

Improved LZ77 Compression with Match-Length-Dependent Sliding Windows^{*}

Yingquan (Cody) Wu[†]

Abstract

We devise and analyze WLZ, a family of LZ77 encoders whose sliding-window sizes depend on match length: short matches use smaller windows and shorter distance fields, while long matches retain access to distant repetitions. Write $B := \log_2 W$ for the maximum window W . A parsing-transfer bound charges both window restrictions and missed matches, preserving established convergence and finite-input minimax orders, and a sharper block charge proves stationary-ergodic universality of exact greedy WLZ when full-window matching begins at length $o(B)$. A calibrated window schedule never increases minimum token cost relative to a single-window baseline and saves $\Omega(BW^{-\beta})$ in rate on specified iid sources, with $\beta > 2$. The remaining gains come from the token code rather than the windows. On a uniform iid source, a growing phrase cap with a cheap length symbol improves redundancy from the $\Omega(\log B/B)$ of the original 1977 fixed-field format, whatever phrase cap it uses, to $O(\log \log B/B)$. Recoding a single-symbol run $(r, 1)$ as $(1, r)$, with a logarithmic-cost count r that may exceed the match cap, codes inputs with $O(n^\alpha)$ runs, $0 \leq \alpha < 1$, in $O(n^\alpha \log n)$ bits, versus $\Omega(n)$ for that format with phrase cap $\Theta(\log W)$ and $W = o(n)$. On globally p -periodic inputs, Huffman coding the fields of a capped parse with nearest-distance ties reduces the large-file rate from $\Theta(B/W)$ under fixed-width coding to at most $3/W$, including tables and framing. Finally, selection among complete WLZ codes yields an entropy-rate estimator consistent almost surely and in mean, with finite-data error bounds for fixed nonuniform iid sources.

1 Introduction

LZ77 represents repeated substrings by backward distances and match lengths [5]. A larger history exposes distant repetitions, but wider fixed-width pointers can make short matches expensive. WLZ assigns a window $W(\ell)$ to each match length ℓ , allowing short matches to use fewer distance bits while longer matches retain access to a larger history (Section 3). The decoder infers the distance range from the decoded length.

^{*}Earlier versions of the window scheme appeared in the author’s 2017 patent and 2021 Data Compression Conference (DCC) report [1, 2]. The codecs WLZ4 and WZIP built on it are described in [3], and their source code is public [4].

[†]Y. Wu is with Tenafe Inc., 591 W. Hamilton Ave, Campbell, CA 95008, USA (e-mail: icodywu@gmail.com).

Practical LZ77 formats differ mainly in how they code these pairs. DEFLATE [6], the format of zlib [7], gzip, ZIP, and PNG, combines a 32 KB window with Huffman coding: literals and match lengths 3–258 share one alphabet, distances use another, and both code a range symbol followed by raw extra bits. This 1990s design remains ubiquitous but trails modern formats in compression ratio and especially in decoding speed: on the Silesia corpus, Zstandard [8] at its fastest level compresses 5% more than zlib at its fastest and decodes about four times as fast [9].

The author’s 2017 patent discusses conditional distance ranges [1, Fig. 3 and eq. (2)], and the conference report [2] introduced the window scheme and an LZ4-based experimental comparison. A separate paper [3] presents two complete codecs built on it, the byte-aligned WLZ4 and the entropy-coded WZIP, with a bounded dynamic-programming parser adapted to length-dependent windows and measurements against LZ4, Zstandard, Brotli and xz; their source code is public [4].

Zstandard and LZ4 [10] are widely deployed lossless codecs in storage and data-processing systems [11, 12], and the top of each class is hard to move. Zstandard at level 22 combines windows up to 128 MiB, binary-tree match finding with dynamic-programming parsing, and FSE entropy coding [8, 13], refined over years of engineering. Its `--max` mode, introduced in version 1.5.7, pursues further compression gains with substantially greater encoding time and memory [14]. On Silesia, the measured alternatives that exceed Zstandard 22’s ratio, Brotli 11 and xz `-9e`, decode 2.6 and 8.8 times more slowly than Zstandard 22 [3]. Against this mature baseline, WZIP levels 11–12, using Huffman coding only, improve the compression ratio over Zstandard 22 by 3.5–3.8% while retaining 88% of its decoding throughput (1148–1150 MB/s); without the x86 and delta filters of its current format, which recode the input before parsing and are independent of the windows, the gain is 1.2–1.4%. WLZ4 improves the ratio over LZ4HC 12 by up to 16.1% while decoding at 3.5 GB/s, 78% of LZ4HC 12’s speed; already at level 6, it exceeds LZ4HC 12’s ratio at three times its compression speed. These Silesia results demonstrate additional compression gains while preserving much of the decoding throughput of the respective baselines [3], motivating the analysis of length-dependent windows and their token codes.

Length-conditioned representations have precedents: Snappy couples a compact short-copy form with an 11-bit offset [15], while LZMA and Brotli use length-derived distance models [16, 17]. These need not impose a unique maximum distance for each length. Lizard does, in a byte-aligned format: it gives 16-bit offsets to matches of 4 bytes or more and reserves 24-bit offsets for matches of 16 or more [18], a two-level length-dependent window. A 2026 GPU-decoding preprint similarly restricts ordinary matches by distance-dependent minimum lengths, with a focus on decoding throughput [19]. Other multiple-index designs include Sadakane and Imai’s two-level hashing with suffix sorting [20], Zstandard’s 2019 double-fast matcher, which uses different seed lengths under a common window limit [21], and the multilayer suffix trees of Crochemore, Langiu, and Mignosi, which keep one sliding-window suffix tree per class of equal offset cost, over geometrically growing windows, to find an occurrence as cheap as the rightmost one [22]. Context-selected dictionaries are a distinct mechanism [23].

Crochemore, Langiu, and Mignosi establish greedy phrase-count optimality under dynamic suffix closure and uniform costs [24]. Minimum-bit parsing accounts for variable costs [25], dictionary-symbolwise flexible parsing attains optimality under dictionary-closure

and coding assumptions [26], and the gap of minimum-bit parsing from greedy encoding can be superconstant [27]. Rightmost occurrence selection and packed-input factorization address complementary construction problems [28, 29, 30].

The original LZ77 proof uses constrained-language phrase counting [5]; later sliding-window and recurrence results establish stationary ergodic optimality [31, 32, 33], and phrase-count bounds under substring complexity give convergence rates [34, 35]. Mixing-based hitting-time results offer quantitative extensions through recurrence-tail bounds [36], and modified LZ schemes achieve $O(1/\log W)$ redundancy under their source assumptions [37].

Here we develop a finite-input analysis under explicit parsing, search, and coding assumptions (Section 2). Separate indexes use seed lengths suited to their match classes; Section 4.4 quantifies their search costs and memory. Parallel search and dictionary-aided compression are discussed in Appendix B.

The analysis rests on one observation: a suffix of a legal copy can keep a distance outside its shorter length’s window, so the suffix closure behind the classical LZ77 comparisons fails. The parsing-transfer inequality (Proposition 5.3) charges this loss, together with matches the parser misses, and makes established phrase-count bounds applicable to WLZ.

Consequently, for arbitrary short-match restrictions below a full-window threshold of $O(\log W)$, exact greedy WLZ keeps the convergence rates of LZ77 under substring-complexity bounds (Corollary 5.7) and, for windows comparable to the input length, its finite-class minimax orders (Corollary 5.10); ordinary LZ77 attains these orders as well. A sharper block charge proves stationary-ergodic universality when the threshold is $o(\log W)$ (Theorem 5.13). The same transfer bounds the cost of capped hash search on average over hash choices, and inspecting the p most recent candidates loses nothing on p -periodic inputs (Corollaries 5.4 and 5.5).

Strict improvements come mainly from the token code. On a uniform iid source, a cheap codeword for a growing phrase cap reduces redundancy from the $\Omega(\log B/B)$ of the original 1977 fixed-field format, whatever cap that format uses, to $O(\log \log B/B)$, where $B := \log_2 W$ (Theorem 6.2 and Proposition 6.3); the comparison isolates the cost of the historical length field, and a single-window encoder with the same token code gains as much. Coding a run of one symbol as $(1, r)$ with a logarithmic count codes inputs with $O(n^\alpha)$ runs, $0 \leq \alpha < 1$, in $O(n^\alpha \log n)$ bits, against $\Omega(n)$ for that format with a logarithmic phrase cap (Proposition 6.6). On globally periodic inputs, Huffman coding the fields of a parse with nearest-distance ties lowers the large-file rate from $\Theta(B/W)$ under fixed-width distances to at most $3/W$, tables and framing included (Theorem 5.6), and Proposition 4.6 bounds the loss of coding range symbols with raw extra bits, as DEFLATE and Zstandard do [6, 8]. The windows themselves never increase the minimum token cost under a calibrated schedule and save a polynomially small rate on a fixed iid source (Theorem 6.5).

As an application, selecting among complete WLZ codes yields an entropy-rate estimator, consistent almost surely and in mean, with finite-data error bounds (Section 7). Appendix D turns to exact parsing of a given input, where the loss of suffix closure reappears: greedy parsing no longer minimizes the number of phrases, and the reduction to maximal matches fails, yet the minimum token cost is computable in $O(n(K + Y) \log n)$ time for K constant-window intervals of lengths and Y constant runs of the length cost.

Taken together, the windows preserve the asymptotic orders and reduce search and memory, since short seeds need only small indexes (Section 4.4), while the larger coding gains

come from the token code. The codec measurements of [3] are consistent with this division: in WLZ4, whose offset fields have fixed widths, the windows raise the compression ratio; in WZIP, ablations indicate that length-conditioned offset coding, the modeling opportunity in (8), improves compression, while restricted windows reduce indexing resources.

2 Background and Reference Parsing Model

Let x_1^n be a sequence over a finite alphabet. A match of length ℓ and backward distance d at position i satisfies

$$x_{i+r} = x_{i-d+r} \quad (0 \leq r < \ell), \quad 1 \leq d < i.$$

Overlapping copies are allowed and are decoded sequentially. A history of W symbols permits distances $1 \leq d \leq W$. The distinction between window sizes (symbols) and distance-field widths (bits) will be maintained throughout: a length- ℓ match has window $W(\ell)$ and width $w(\ell) := \log_2 W(\ell)$, and the maximum window W has width $B := \log_2 W$.

We write X_1^n for a random input, x_1^n for a realization, and P_n for the law of X_1^n . Lowercase symbols denote local indices and match parameters, such as i, ℓ, d ; capacities and counts usually use capitals, such as W, L, N, M , with n retained for input length. Total bit costs use C , local coding costs use a, b, c , coding rates use $\mathcal{R}$, and the entropy rate uses h . Calligraphic letters also distinguish sets and families. Header and framing costs use Γ , while H is retained for entropy and Huffman annotations. Descriptive labels, such as cap and cut, are upright. We use $:=$ to introduce a quantity by a formula; $=$ is retained for identities, assumptions, parameter choices, and recalled relations.

For the theoretical reference parser, the set of candidate starts is the entire decoded history within the allowed distance range. At each position, the parser chooses a longest admissible match of at least the supported minimum length; if none exists it emits one literal. It then advances by the chosen phrase length. For a finite input, matches are truncated at the end of the input. For the reference model, use the cheaper of a chosen match token and its individual literals, retaining the same conceptual phrase boundary. This convention caps the cost of a length- ℓ phrase by $b\ell$, where b is the literal cost, without changing the greedy boundaries.

A chained hash table accelerates candidate generation: a head table gives the most recent inserted position for each hash value, and a chain links earlier positions with the same hash. Section 3 describes its layout.

Exact byte comparison is required to distinguish genuine seed matches from collisions. To implement the reference search, every eligible source position, including positions skipped by a match, must be inserted or otherwise recoverable, and the chain must be searched without discarding candidates needed for the longest admissible match.

Practical encoders often cap the number of candidates, retain only the latest hash entry, skip insertions, or apply lazy matching. These are useful speed choices, but their output is not assumed to equal the reference parse. The analysis below either explicitly requires exact greedy parsing or charges missed and declined matches. None of the asymptotic statements is inferred solely from a hash table’s seed length. Figure 1 shows how greedy, lazy, and

history `abc_bcdefg_efgh_pqr_qrst_stuvw_`, then the input

	a	b	c	d	e	f	g	h	p	q	r	s	t	u	v	w
greedy	abc			defg			h	pqr		stuvw					89 bits	
lazy	a	bcdefg					h	p	qrst			uvw			87 bits	
minimum cost	a	bcdefg					h	pqr		stuvw					78 bits	

Figure 1: Three parses of one input with a single window, literals of 9 bits (shaded) and matches of 20 bits. Greedy parsing takes the longest match at each position. Lazy parsing first emits a literal when the next position offers a longer match: it gains on `abcdefgh` but loses on `pqrstuvw`, where deferring to `qrst` costs a third phrase. The minimum-cost parse, a shortest path over all phrases, makes the better choice in each half.

minimum-cost parses of one input differ; minimum-cost parsing returns in Section 4 and Appendix D.

LZSS permits literals as an alternative to a match [38]. DEFLATE uses a combined literal/length alphabet and a separate distance alphabet, each with its specified code and extra bits [6]. LZ4 and Zstandard use different token layouts and entropy-coding choices [10, 8]. These distinctions must be held fixed or charged when attributing a coding gain to the window schedule.

Throughout, the results concern specified parsers and complete codes. Parsing is exact greedy or minimum-bit unless stated otherwise; heuristic search enters only through explicitly charged failures. Token costs are fixed unless fields are entropy coded, in which case tables and framing are charged. Headers, schedules, and initialization are included in code lengths, and large-file rates take the input-length limit before the window limit. Upper bounds hold for the stated source classes and formats. They are not claimed to be best-known redundancy orders or rates uniform over all stationary ergodic sources [39], and they do not by themselves certify a heuristic implementation. Lower bounds hold in their stated coding models.

3 Match-Length-Dependent Sliding Windows

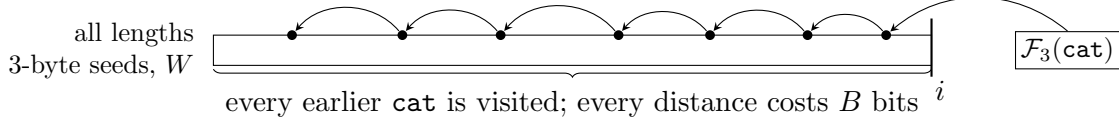
Let $\mathcal{F}_k$ hash a seed of k bytes. The hash-seed length must satisfy $k \leq \ell$ for every match length ℓ served by the index, so a genuine match agrees on all hashed bytes. Table 1 illustrates a length-dependent schedule and one index per length class. The length is decoded before the distance, so the distance-field width is determined without an extra class selector when the schedule is shared. For a match cost $a(\ell) + w(\ell)$ and literal cost b per byte, the match is beneficial relative to its literals exactly when $a(\ell) + w(\ell) < b\ell$. A small window alone is therefore not a complete compressibility criterion.

Short and long matches affect the search differently. For example, many candidate strings may share the seed `cat` while relatively few share `cat and`. An index using a longer seed can avoid visits to candidates that cannot produce a long match, and an index for short matches need only retain a short window (Figure 2). This is a source-dependent improvement, not a

Table 1: Illustrative length-dependent windows and hash seeds.

Match length ℓ (bytes)	2	3	4	5	6	7	≥ 8
Distance width $w(\ell)$ (bits)	5	9	12	15	17	19	21
Hash seed (bytes)	2	3	4	5	6	7	8

(a) a single window



(b) length-dependent windows

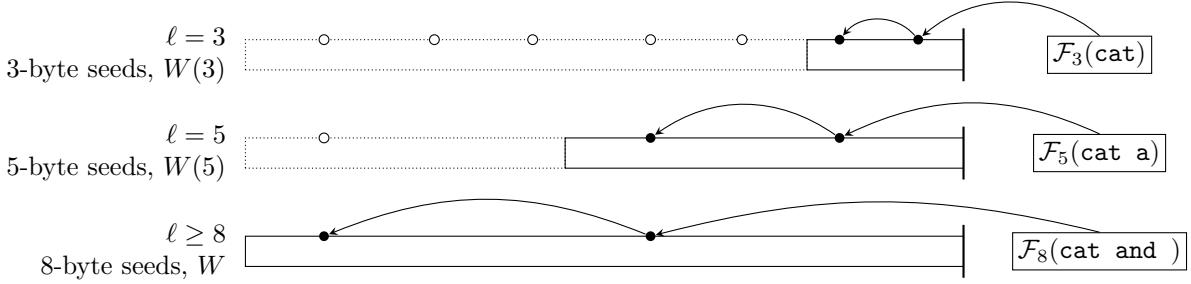


Figure 2: Hash-chain search with one window and with length-dependent windows (not to scale). Bars show the history before the current position i ; dots mark earlier occurrences of each index’s seed, and arrows follow its chain from the bucket head, nearest first. (a) One index on 3-byte seeds visits every earlier `cat` within W . (b) One index per length class, keyed by the class’s minimum length and retaining only its window. Short seeds have many candidates but only within a short window; hollow dots lie beyond it and are never visited. The 8-byte seed `cat and` reaches the whole history with few candidates.

worst-case exponential speedup. The candidate-count analysis in Section 4 separates genuine seed matches from hash collisions.

Each index is a chained hash table. For an index with window V , the chain is a circular array of exactly V entries: the link of source position j occupies slot $j \bmod V$, the bitwise $j \wedge (V - 1)$ when V is a power of two, and the slot is reused V positions later, when j leaves the window. A dedicated length- ℓ index thus uses $W(\ell)$ link entries, a shared index retains the largest window it serves, and the head table is sized separately. In WZIP and WLZ4 [4], the link of j stores the distance back to the previous position with the same hash, replaced by a sentinel that ends the search when that position lies outside the window. A search follows the chain from the head entry, accumulating these distances, and stops once their sum leaves the window. Every slot it reads therefore belongs to a position inserted within the window and not yet reused, so the chain array needs no initialization, only the head table does; no input pays for clearing V entries. Links may instead store absolute positions or other age information, provided expired candidates are rejected before recycled slots are read. When distance V is admissible, search at position i precedes insertion of i , which reuses the slot of the oldest eligible position $i - V$.

Each fixed short-length class ℓ traverses eligible candidates in most-recent-first order, verifies all ℓ symbols to reject hash collisions, and stops its chain at the first verified match. It returns length ℓ with the nearest retained eligible distance. Only the largest-window class extends candidates beyond its seed length and continues searching for the longest permitted match, subject to any inspection cap. With an ordinary match cap W and nearest-distance ties, it can also stop at its first verified admissible length- W match: no later candidate can improve its length or distance. Retaining the nearest candidate among equal-length matches gives nearest-distance ties. The encoder selects among the classes' candidates; success in one short class does not terminate the other searches. For variable-cost distances, the nearest match need not have minimum encoded cost.

Adjacent length classes may share an index whose seed does not exceed the minimum length it serves. Its chain must retain the largest window served by that index. Table 2 gives a consistent shared-index variant. Each served length has its own completion condition: a length-3 hit does not end a shared length-3/4 search before length 4 is resolved. In particular, an index serving a 19-bit distance range requires that range of retention for exact search.

Table 2: A shared-index variant of Table 1.

Length (bytes)	Distance width (bits)	Seed (bytes)	Chain width (bits)
2	5	2	—
3	9	3	12
4	12	3	12
5	15	5	19
6	17	5	19
7	19	5	19
≥ 8	21	8	21

The chain-free length-2 option is a heuristic unless its direct lookup identifies the exact seed, or an equivalent collision-safe representation is used. Whether to include length-2 matches depends on their net coding savings and the associated search and memory costs.

Parallel execution of the index searches and preset-dictionary addressing are described in Appendix B.

4 Theoretical Analysis

The analysis distinguishes the admissible matches, their encoded lengths, and the algorithm used to find them. The original LZ77 analysis uses distinct-phrase counting for constrained sources [5, Sec. III-C]. We adapt this elementary argument to length-dependent windows, then quantify coding gains and search costs. Throughout the paper, $\log$ denotes the base-two logarithm and $\ln$ the natural logarithm.

Let x_1^n be the input. A length- ℓ match at position i and backward distance d is admissible if

$$1 \leq d \leq \min\{i-1, W(\ell)\}, \quad x_{i+t} = x_{i-d+t} \quad (0 \leq t < \ell).$$

Here $W(\ell)$ is a window size measured in input symbols, and $w(\ell) := \log W(\ell)$ is an integer distance-field width. Overlapping copies are allowed and decoded sequentially. We initially

assume fixed token costs: a literal costs b bits and a match costs

$$c_W(\ell, d) := a(\ell) + w(\ell), \quad (1)$$

where $a(\ell)$ includes a match discriminator and a prefix code for the length. The distance $d - 1$ is represented in $w(\ell)$ bits. Decoding the length determines the window without a separate class identifier. Any schedule description and framing are counted separately as Γ_n . For Huffman-coded distances, the actual distance-symbol and extra-bit costs must replace $w(\ell)$.

4.1 Counting short phrases with different windows

For the following lemma the parser chooses a longest admissible match at each phrase start, or one literal if no match exists. All eligible past positions are searched. The cheaper literal representation is used whenever needed, retaining the conceptual phrase boundary. Let $D_m(x)$ be the number of distinct length- m substrings of x_1^n , and let S_m count conceptual phrases shorter than m .

Lemma 4.1 (Spacing of short phrases) *For $n \geq m$, with m at least the minimum match length and no larger than the maximum supported match length,*

$$S_m \leq D_m(x) \left(1 + \left\lfloor \frac{n - m}{W(m) + 1} \right\rfloor \right) + m - 1. \quad (2)$$

Proof. Consider two short-phrase starts $i < j$ with complete length- m continuations. If these continuations agree and $j - i \leq W(m)$, an admissible length- m match exists at j , contradicting selection of a shorter phrase. Starts with the same continuation are therefore separated by at least $W(m) + 1$. Their number is at most $1 + \lfloor (n - m) / (W(m) + 1) \rfloor$. There are $D_m(x)$ possible continuations, and at most $m - 1$ phrase starts without a complete continuation. Summing proves the claim. $\square$

Let r_m denote the worst per-symbol cost over supported lengths at least m :

$$r_m := \sup_{\substack{\ell \geq m \\ \ell \text{ supported}}} \frac{\min\{b\ell, a(\ell) + w(\ell)\}}{\ell}.$$

Corollary 4.2 (Finite-input rate bound) *Encode x_1^n by exact longest-match parsing, representing each chosen match by the cheaper of its token and its literal expansion. For every supported ordinary match length $m \leq n$, the complete code length satisfies*

$$\frac{C_W(x) - \Gamma_n}{n} \leq r_m + \frac{b(m - 1)}{n} \left[D_m(x) \left(1 + \left\lfloor \frac{n - m}{W(m) + 1} \right\rfloor \right) + m - 1 \right]. \quad (3)$$

Proof. Long conceptual phrases cost at most nr_m bits in total. Short phrases contain at most $(m - 1)S_m$ symbols and cost at most $b(m - 1)S_m$ bits. Apply Lemma 4.1 to S_m . $\square$

For imperfect search, let E_m count short-phrase starts having a complete continuation and an admissible length- m match that was missed or declined. Deleting those starts restores the

spacing proof. Therefore (2) holds with an additional E_m , and (3) holds with an additional $b(m-1)E_m/n$. This explicitly accounts for capped searches, incomplete insertion, and other parsing decisions.

Motivated by the constrained-source framework of Ziv and Lempel [5, Sec. III-A], assume $D_m(x_1^n) \leq A2^{hm}$ for all $1 \leq m \leq n$, with A and $h > 0$ fixed independently of m, n . Consider windows bounded by 2^B , with $W_B(\ell) = 2^B$ for all $\ell \geq t_B$, where $t_B = o(B)$. Assume unbounded supported lengths, $a(\ell) \leq a_0 + a_1 \log(\ell + 1)$, and $\Gamma_n = o(n)$ for fixed B . Taking $m = \lfloor (B - 3 \log B)/h \rfloor$ in (3) yields

$$\limsup_{n \rightarrow \infty} \frac{C_{W_B}(x_1^n)}{n} \leq h + O\left(\frac{\log B}{B}\right). \quad (4)$$

Indeed, the short-phrase contribution is $O(B^{-2})$, and $r_m \leq [B + a_0 + a_1 \log(m+1)]/m$. This preserves a classical redundancy order under a language-growth assumption.

4.2 Compression savings and window selection

Let a comparator use one window 2^B , the same literal cost, and match cost $a(\ell) + B$. Fix any comparator parse P . Partition its matches into $\mathcal{A}$, admitted by the length-dependent windows, and $\mathcal{F}$, excluded by them. Define

$$G(P) := \sum_{(\ell, d) \in \mathcal{A}} [B - w(\ell)], \quad R(P) := \sum_{(\ell, d) \in \mathcal{F}} [b\ell - a(\ell) - B].$$

Lemma 4.3 (Reference-parse comparison) *Let $C_W^*(x)$ be the minimum token cost of an admissible parse. Then*

$$C_W^*(x) \leq C_0(P) - G(P) + R(P). \quad (5)$$

Proof. Keep all admitted matches and original literals, and replace every excluded match with its literals. All later phrases see the same decoded history. The resulting admissible representation has exactly the cost on the right side. Its cost upper-bounds the optimum. $\square$

If all comparator matches are beneficial, $R(P) \geq 0$. For a heuristic encoder with excess token cost $E_W := C_W^{\text{heur}} - C_W^*$ and header difference $\Delta\Gamma := \Gamma_W - \Gamma_0$, a sufficient condition for strict improvement is

$$G(P) > R(P) + E_W + \Delta\Gamma. \quad (6)$$

Shorter distance fields alone therefore do not guarantee improvement. Exact minimum-bit parsing is a shortest-path problem with an edge for every admissible phrase [25]. Appendix D shows which single-window shortcuts fail under length-dependent windows and computes C_W^* exactly in $O(n(K+Y) \log n)$ time, for K constant-window intervals of lengths and Y constant runs of the length cost.

The same comparison gives a rule for choosing window widths. Freeze N_ℓ reference matches of length ℓ , and let $A_\ell(w)$ count those with distance at most 2^w . Retaining admitted matches and literalizing excluded matches costs

$$F_\ell(w) := A_\ell(w)[a(\ell) + w] + [N_\ell - A_\ell(w)]b\ell.$$

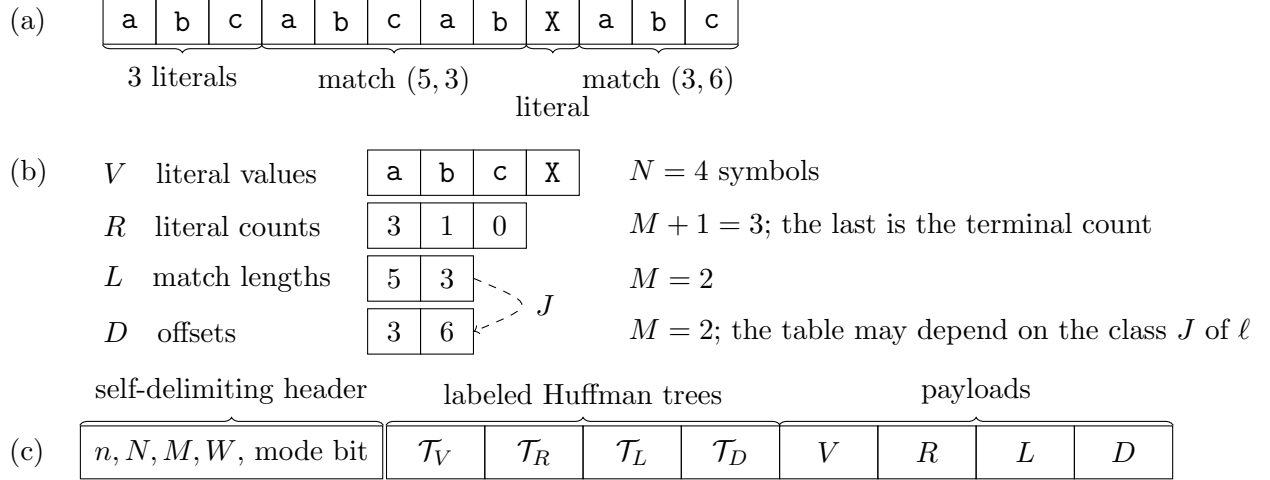


Figure 3: Fields of a parse and the complete format of Section 4.3. (a) The greedy parse of `abcabcabXabc` with nearest-distance ties: three literals, the overlapping match (5, 3), a literal, and the match (3, 6). (b) Its four field streams. The decoder reads a length before its offset, so an offset table may be selected by the length class J . (c) The complete format: a self-delimiting header, four labeled Huffman trees, and the four payloads, whose known sizes $N, M + 1, M, M$ delimit them.

Proposition 4.4 (Marginal window expansion) *With $\Delta A := A_\ell(w + 1) - A_\ell(w)$,*

$$F_\ell(w + 1) - F_\ell(w) = A_\ell(w) - \Delta A[b\ell - a(\ell) - w - 1]. \quad (7)$$

Proof. Every previously admitted match incurs one more bit. Each newly admitted match saves $b\ell - a(\ell) - w - 1$ bits relative to literals. $\square$

Thus window expansion helps exactly when the newly admitted savings exceed the additional cost on existing matches. Nondecreasing widths over J length classes can be selected by

$$T(j, w) := F_j(w) + \min_{v \leq w} T(j - 1, v),$$

where class objectives sum the relevant length objectives. Prefix minima give $O(J(B_{\max} + 1))$ time after histogram construction. This optimizes a fixed trace, not joint window selection and reparsing. Schedule metadata must be charged when widths are selected per block.

4.3 Entropy coding of the token fields

Entropy and Huffman cost analysis of LZSS fields has a long history: Bell studied lengths, distances, literals, and flags [40, Sec. 6.3], and Fenwick analyzed entropy-coded lengths and distance precisions with raw distance bits [41, Secs. 5 and 8]. We recall the coding bounds needed to connect the specified WLZ parses to complete encoded lengths. Figure 3 shows the fields of a parse and the complete format used below.

For a fixed parse with M matches, let J be the decoded length class and Z the distance symbol. The ideal empirical saving from conditioning is

$$M\hat{H}(Z) - M\hat{H}(Z | J) = M\hat{I}(Z; J). \quad (8)$$

The decoded length identifies J without a class selector. Unconstrained binary Huffman coding changes the saving by $\delta_g - \delta_c$, with each rounding term in $[0, M)$ when singleton alphabets may use zero bits. Extra distance bits, table headers, and format restrictions must also be charged. This standard identity explains the modeling opportunity seen in LZMA and Brotli [16, 17]. It does not by itself give a net saving or a benefit from excluding matches. Measurements are consistent with this: with the WZIP codec’s bounded dynamic-programming parser, replacing its per-length-group offset tables by one table lowers the compression ratio on the Silesia corpus by 0.58%, whereas admitting every length at every distance, with offsets still coded per group, leaves it unchanged [3]. Each ablation reparses the input, so it indicates rather than isolates the effect of conditioning.

The complete field streams also permit a useful payload comparison. Group consecutive literals by a count before each match, as in Zstandard [8, Sec. 3.1.1.3.2]. Our model also includes an explicit terminal count, possibly zero; Zstandard appends the remaining literals after its final sequence. In the following proposition, fitted code tables, including their integer symbol labels, are supplied to the decoder without charge. Match lengths and offsets are coded as complete values. Write C_H^{pay} for the resulting Huffman payload, excluding table descriptions and stream framing.

Proposition 4.5 (Separate-field Huffman payload) *Fix a valid parse with N literals and M matches, and put $P := N + M$. Let V, R, L, D be its literal-value, literal-count, match-length, and offset streams, of respective sizes $N, M + 1, M, M$. Define*

$$\mathcal{E}_4 := N\hat{H}_0(V) + (M + 1)\hat{H}_0(R) + M\hat{H}_0(L) + M\hat{H}_0(D), \quad (9)$$

where empty streams contribute zero. Unconstrained empirical binary Huffman coding satisfies

$$C_H^{\text{pay}} \leq \mathcal{E}_4 + N + 3M + 1, \quad (10)$$

$$C_H^{\text{pay}} \leq b_0N + P + 1 + 2Mb_M = O(P \log(P + 1)), \quad (11)$$

where b_0 is a fixed valid literal width and $b_M := \max\{1, \lceil \log \max\{1, M\} \rceil\}$. The order assumes a fixed literal alphabet.

Proof. The empirical Huffman bound charges at most one bit above entropy per stream symbol, giving (10). For the second bound, fixed-width literal coding is a valid competitor. Unary count words 0^r1 cost $\sum_j (r_j + 1) = N + M + 1 = P + 1$ bits. Each match-field stream has at most M distinct observed values, so a b_M -bit index is a valid competitor for each value. Huffman optimality gives (11). $\square$

No independence of the streams is needed. Supplying the integer labels free is essential to the payload support bound: coding a range identifier followed by raw extra bits requires charging those bits.

For $n \geq 1$ over a fixed alphabet Σ , a complete format transmits n, N, M, W self-delimitingly, followed by the four labeled Huffman trees and then the four field payloads. A fixed mode bit distinguishes lengths capped at W from unrestricted lengths. Each tree has an empty-tree flag; a nonempty tree is described by preorder internal/leaf flags and a

fixed-width label at each leaf. Let K_s be the observed support size in field s , with label universes

$$U_V := |\Sigma|, \quad U_R := N + 1, \quad U_D := W, \quad U_L := \min\{n, W\} \text{ or } n,$$

according to the mode. Positive lengths and distances are labeled from zero. A nonempty tree has $2K_s - 1$ nodes, so the complete length C_H satisfies

$$C_H - C_H^{\text{pay}} \leq O(\log(n + 1) + \log(W + 1)) + \sum_{s \in \{V, R, L, D\}} K_s(2 + \lceil \log U_s \rceil). \quad (12)$$

The constant covers empty-tree flags. The known stream sizes $N, M + 1, M, M$ delimit the payloads, including zero-bit singleton codes, so this is a prefix format across input lengths. Frequencies, search indexes, and the window schedule are not needed for reconstruction from complete integer fields. Any additional decoder parameters must be fixed or charged separately.

These constructions encode a fixed parse; refitting tables over alternative parses is not covered by the fixed-cost parsing proofs. A one-bit choice between the complete fitted format and a qualifying original complete prefix code C_0 gives length $1 + \min\{C_0, C_H\}$ and preserves each applicable original upper bound within one bit.

Zstandard uses finite state entropy (FSE) coding, which carries a coding state between symbols, for its literal-length, match-length, and offset codes, interleaved with raw extra bits, and Huffman coding for its literals [8, Secs. 3.1.1.3 and 4.1]. The refinements below are proved for Huffman coding; transferring them to FSE requires controlling its normalization, state termination, and extra bits.

DEFLATE and Zstandard reduce their coding alphabets by coding range identifiers and appending prescribed raw bits [6, Sec. 3.2.5], [8, Sec. 3.1.1.3.2.1]. The following specialization of established monotone-alphabet grouping analysis [42, 43] quantifies when this representation retains the benefit of coding complete values.

Proposition 4.6 (Significant-bit coding with an irregular head) *Fix an integer $\nu \geq 1$ and put $T := 2^\nu$. Let $X \in \{0, \dots, c\}$, where $c \geq T$. Give every $x < T$ and the cap $x = c$ an exact symbol. For $T \leq x < c$, encode the label (r, u) followed by r raw bits, where*

$$r := \lfloor \log x \rfloor - \nu + 1, \quad u := \lfloor x/2^r \rfloor.$$

Thus ν leading significant bits are retained. All labels, including exact symbols, share one coding alphabet. Suppose $p_k \geq \dots \geq p_{c-1}$ for some $1 \leq k < T$, where $p_x := \Pr(X = x)$; the head and cap probabilities are unrestricted. For the label J , raw width r_J (zero for exact symbols), and $\tau := \Pr(k \leq X < c)$, the grouping loss satisfies

$$0 \leq \Delta := H(J) + \mathbb{E}r_J - H(X) \leq \frac{2\tau}{e \ln 2 (2^\nu - k + 1)}. \quad (13)$$

With optimal unrestricted Huffman codes for this same distribution, the mean lengths obey $0 \leq L_{\text{group}} - L_{\text{whole}} < 1 + \Delta$. An unused cap may have probability zero.

The proof is in Appendix C. For fixed k , the grouping loss is $O(2^{-\nu})$; the same order holds for growing k if $2^\nu \geq 2k$. Applied to the empirical distributions of R, L, D satisfying these conditions, with the literal stream unchanged,

$$C_{\text{group}}^{\text{pay}} - C_{\text{H}}^{\text{pay}} \leq 3M + 1 + (M + 1)\Delta_R + M\Delta_L + M\Delta_D. \quad (14)$$

No field independence is required. With common precision, this adds $O((M+1)2^{-\nu})$ grouping cost beyond $O(M+1)$ Huffman rounding, preserving rate orders that already allow such rounding. Table costs are excluded.

The construction has $O(2^\nu[1 + \log(c/2^\nu)])$ range labels: at most $2^{\nu-1}$ per binary scale, plus the exact head and cap. For fixed precision, the alphabet shrinks from $c+1$ values to $O(\log c)$ symbols, with smaller frequency tables and Huffman trees, while raw suffixes need only bit extraction [42]. DEFLATE uses these ranges for $D-1$ with $\nu=2$ and $L-3$ with $\nu=3$, including an exact symbol for $L=258$ [6, Sec. 3.2.5]; for tails decreasing from $k=1$, the grouping losses are below 0.266 and 0.133 bits per value. Zstandard's literal-count, shifted-length, and coded-offset partitions refine dyadic ranges [8, Sec. 3.1.1.3.2.1.1], so the same argument bounds each grouping loss by $1/(e \ln 2) < 0.531$ bits per value when the positive-value distribution is nonincreasing.

4.4 Hash-chain search and memory

Fix a query seed of length k , $N \leq W$ stored seeds, and let M_k of those seeds equal the query. Choose a hash function, independently of the input, from a family whose collision probability for unequal seeds is at most $1/K$, where K is the number of buckets. The subscript hash on an expectation denotes averaging over the random hash choice; X additionally denotes averaging over the input.

Theorem 4.7 (Expected bucket size) *If Q_k is the number of prior nodes in the query bucket, then*

$$\mathbb{E}_{\text{hash}} Q_k \leq M_k + \frac{N - M_k}{K}. \quad (15)$$

For an iid source with symbol probabilities p_a and collision entropy $H_2 := -\log \sum_a p_a^2$, at a fixed input position,

$$\mathbb{E}_{X, \text{hash}} Q_k \leq \min(W, k-1) + W2^{-kH_2} + \frac{W}{K}. \quad (16)$$

Proof. Equal seeds collide with probability one and unequal seeds with probability at most $1/K$. Summing indicator expectations proves (15). Two nonoverlapping iid seeds agree with probability $(\sum_a p_a^2)^k = 2^{-kH_2}$. At most $k-1$ earlier seeds overlap the query, each contributing at most one to the expected exact-match count. Substitution gives (16). $\square$

The dedicated fixed-length searches admit a sharper work bound. For a class with seed and target length both ℓ and history $W(\ell)$, let R_ℓ count nodes inspected before the first verified match, the bucket end, or an inspection cap S_ℓ . At a fixed input position, let N count its eligible retained positions, of which M_ℓ have the same length- ℓ seed. Under the same input-independent hash assumption, with K_ℓ buckets,

$$\mathbb{E}_{\text{hash}} R_\ell \leq \min \left\{ S_\ell, \mathbf{1}_{\{M_\ell > 0\}} + \frac{N - M_\ell}{K_\ell} \right\} \leq \min \left\{ S_\ell, 1 + \frac{N}{K_\ell} \right\}.$$

Indeed, at most one equal-seed node is inspected; every other inspected node is an unequal-seed hash collision. Summing collision indicators over the retained positions proves the bound, without a source model or independence among collisions. Taking K_ℓ proportional to $W(\ell)$ gives $O(1)$ expected candidate inspections per short class, including highly repetitive inputs. For example, if all retained seeds equal the query, first-match stopping visits one node instead of the entire bucket. Equality verification costs up to $O(\ell)$ per node; hash computation and index updates must also be counted. This bound applies to dedicated full-length seeds. In the largest-window class, a seed match shorter than the ordinary cap still requires extension and does not itself justify stopping.

Shorter seeds also permit smaller windows at a comparable probability of finding an exact seed match. For a uniform iid source on $q \geq 2$ symbols, fix a query position with W_k preceding symbols and retain and search every eligible occurrence. Lemma 6.1 gives

$$\Pr\{\text{no exact } k\text{-symbol seed match within distance } W_k\} \leq 2 \exp\left\{-\frac{(\ln 2)W_k}{8q^k}\right\}.$$

Indeed, conditional on the query word, the preceding block is independent of it, and any occurrence wholly inside that block is admissible. Thus, for $0 < \varepsilon < 1$, choosing

$$W_k \geq \frac{8}{\ln 2} q^k \ln \frac{2}{\varepsilon}$$

makes the miss probability at most ε . At fixed confidence, this sufficient window size decreases by a factor q when the seed is shortened by one symbol. This formalizes the compensation between a shorter window and a higher exact seed-match probability in this source model. Conversely, for a random uniform-iid query at a fixed position, a union bound over distances (including overlaps) gives seed-match probability at most $\min\{1, W_k q^{-k}\}$. Coverage at least $1 - \varepsilon$ therefore requires $W_k \geq (1 - \varepsilon)q^k$. Thus q^k is the optimal window order for fixed confidence; this necessity is not conditional on each specified query word and does not give matching dependence on vanishing ε . For a class indexed by a seed shorter than its target match length, the guarantee concerns seed discovery; extension must still be verified.

For a nonuniform iid source, keeping $W_k 2^{-kH_2}$ comparable instead balances the nonoverlapping term in the expected candidate-count bound (16). It does not give the same coverage guarantee for every query word. Higher exact seed-match probability should also be distinguished from accidental hash collisions: only the former supplies useful candidates. Frequent exact matches can help an existence search stop early, but increase the nodes in an exhaustive bucket scan.

Increasing the seed length can therefore offset a larger window's genuine candidate count, while the bucket count must control spurious collisions. These are candidate-node bounds, not worst-case time bounds: constant strings can have all seeds equal, and match extension and index maintenance also cost time. Summing over all input positions gives a conservative whole-file bound without assuming the fixed-position calculation at parser-selected starts.

With index groups g , chain windows V_g , bucket counts K_g , and entry widths s_g, t_g in bytes, encoder memory includes

$$W_{\max} + \sum_g (s_g V_g + t_g K_g)$$

plus lookahead, token buffers, entropy tables, and allocation overhead. The circular layout in Section 3 uses exactly V_g chain-link entries per index. Modulo addressing fixes this entry count; s_g still accounts for the representation of each link. The decoder uses one history buffer of size $W_{\max}$. Geometrically increasing chain windows with ratio at least $r > 1$ have total chain-entry count at most $rW_{\max}/(r-1)$. If also $K_g = O(V_g)$ uniformly over the indexes, the head and chain arrays together have $O(W_{\max})$ entries; their byte cost still depends on s_g and t_g . These quantities should accompany comparisons with prior accelerated LZ77 matching [20].

5 Convergence and Finite-Input Optimality

An asymptotic statement requires a family of encoders, not a fixed window table. Put $W = 2^B$ and let $B \rightarrow \infty$. We assume that all supported lengths $\ell \geq t_B$ have the full window W , that shorter windows do not exceed W , and that t_B is at least the minimum match length. The exact greedy family supports every length from that minimum through at least W ; when explicitly stated below, lengths are unbounded. With b, a_0, a_1 fixed independently of B, n , let

$$a(\ell) \leq a_0 + a_1 \log(\ell + 1), \quad a_0, a_1 \geq 0, \quad \mathcal{R}_B(x) := \limsup_{n \rightarrow \infty} \frac{C_B(x_1^n)}{n}.$$

Headers have $\Gamma_n = o(n)$ for each fixed B . Literal fallback retains conceptual greedy boundaries as defined in Section 2. The order of limits is first $n \rightarrow \infty$ and then $B \rightarrow \infty$; Corollary 5.8 gives a joint finite-input bound, and Section 7.2 treats data-dependent selection among a growing set of windows.

There are two different convergence scales. For a positive-entropy source, one studies redundancy relative to its entropy, often in powers of $1/B = 1/\log W$. For a zero-entropy or repetitive source class, the output rate itself can decay as a power of $1/W$.

We first sharpen the elementary counting bound. For a supported threshold m with $W_B(m) = W$, define

$$K_m := \max\{b, B + a_0 + a_1 \log(m + 1)\}.$$

Each conceptual phrase shorter than m costs at most K_m ; each longer phrase costs at most K_m/m per symbol, because both $1/\ell$ and $\log(\ell + 1)/\ell$ decrease for $\ell \geq 1$. Therefore Lemma 4.1 gives

$$\frac{C_B(x_1^n) - \Gamma_n}{n} \leq \frac{K_m}{m} + \frac{K_m}{n} \left[D_m(x_1^n) \left(1 + \left\lfloor \frac{n - m}{W + 1} \right\rfloor \right) + m - 1 \right]. \quad (17)$$

If $D_m(x_1^n) \leq P(m)$ uniformly in n , this implies

$$\mathcal{R}_B(x) \leq K_m \left(\frac{1}{m} + \frac{P(m)}{W + 1} \right). \quad (18)$$

Charging short phrases by tokens rather than by their literal lengths is essential when m is much larger than B . For $P(m) = Am^s$ and $t_B = O(B)$, choosing $m = \lfloor W^{1/(s+1)} \rfloor$ gives $\mathcal{R}_B = O(BW^{-1/(s+1)})$. The same bound gives the following rate directly.

Proposition 5.1 (Stretched-exponential substring complexity) *Assume $D_m(x_1^n) \leq A2^{cm^\alpha}$ uniformly in $n \geq m$, for fixed $A, c > 0$ and $0 < \alpha < 1$. Under the same exact greedy family with $t_B = O(B)$,*

$$\mathcal{R}_B(x) = O(B^{-(1/\alpha-1)}). \quad (19)$$

Proof. Choose a fixed $q \geq 1/\alpha$ and set $m = \lfloor ((B - q \log B)/c)^{1/\alpha} \rfloor$ for sufficiently large B . Then $m \geq t_B$, $m \leq W$, $K_m = O(B)$, and $P(m)/W \leq AB^{-q}$. Equation (18) gives $O(B^{1-1/\alpha}) + O(B^{1-q})$, proving the result. $\square$

Here the exponent of $1/\log W$ exceeds 1 when $\alpha < 1/2$; for instance, $\alpha = 1/3$ gives $O((\log W)^{-2})$ under this zero-entropy complexity assumption. For comparison, the exponential envelope $D_m \leq Am^s 2^{hm}$ with $h > 0$, a language-growth bound, gives only $\mathcal{R}_B \leq h + O(\log B/B)$ by this argument: take $m = \lfloor [B - (s+2) \log B]/h \rfloor$ and assume $t_B = o(B)$.

Table 3: Convergence guarantees for the stated exact greedy family. Constants in the source envelopes are fixed independently of W, n .

Substring-complexity envelope	Output-rate guarantee
$D_m \leq Am^s$, fixed $s > 1$	$O((\log W)W^{-1/s})$
$D_m \leq Am$	$O((\log W)^2/W)$
$D_m \leq A2^{cm^\alpha}$, $0 < \alpha < 1$	$O((\log W)^{-(1/\alpha-1)})$
$D_m \leq Am^s 2^{hm}$, $h > 0$	$\leq h + O(\log \log W / \log W)$

Table 3 collects these guarantees with the power-law rates of Corollary 5.7. For power-law envelopes, the transfer below increases the exponent from $1/(s+1)$ to $1/s$ for $s \geq 1$.

5.1 Transfer from a reference block parse

Set $K_W := \max\{b, B + a_0 + a_1 \log(W+1)\}$. Conceptual phrases shorter than W cost at most K_W each; longer phrases cost at most K_W/W per source symbol. The following comparison accommodates the fact that the length-dependent dictionary need not be closed under taking suffixes. For example, in `abcdabcd`, the second `abcd` can use length four and distance four when $W(4) = 4$, whereas its `cd` suffix cannot use that distance when $W(2) = 2$.

Theorem 5.2 (Block-parse transfer) *Assume $1 \leq t_B \leq W$ and the exact greedy model above. Partition x_1^n into consecutive blocks of length W , with a final shorter block. Let $Z_W(x_1^n)$ be the sum of the ordinary greedy LZ77 phrase counts of these blocks, each parsed independently with every earlier position in that block available and with overlapping copies allowed. Then*

$$C_B(x_1^n) - \Gamma_n \leq \frac{K_W}{W}n + [K_W + b(t_B - 1)]Z_W(x_1^n). \quad (20)$$

Proof. Consider a reference copy occupying positions $[p, q]$. Its backward distance is less than W . At any new-parser phrase start $i \in [p, q]$ with $q - i + 1 \geq t_B$, the suffix x_i^q has a legal full-window copy at that same distance. Its length is at most W and is supported. Exact

greedy parsing must reach at least $q + 1$, so i is the last new phrase start in this reference phrase. Consequently, every start except the last lies in its final $t_B - 1$ positions. The corresponding nonlast conceptual phrases end before the last start, and together consume at most $t_B - 1$ symbols. Literal fallback bounds their combined cost by $b(t_B - 1)$. If the last new phrase has length less than W , it costs at most K_W ; otherwise it is charged below. A reference literal contains at most one new start and satisfies the same upper bound.

Charge each new phrase shorter than W to the reference phrase containing its start. The charge is at most $K_W + b(t_B - 1)$ per reference phrase. All remaining new phrases cost at most K_W/W per consumed symbol. Summing gives (20). $\square$

The block boundaries are used only for comparison; the new parser need not reset its history. In particular, the theorem applies to exact greedy parsing and does not require a minimum-bit parser. With an imperfect parser, the suffix-reaching step in this proof must be established or its failures charged separately.

The comparison can retain the individual distance thresholds and charge search errors separately. The following statement also specifies what an approximate search must preserve.

Proposition 5.3 (Schedule-sensitive transfer with search failures) *Retain the window and code assumptions above, with minimum match length m_0 , every length from m_0 through W supported, and $m_0 \leq t_B \leq W$. Fix the reference block parse P of Theorem 5.2, with $Z := Z_W(x_1^n)$ phrases and reference-copy lengths and distances (ℓ_j, d_j) . Define*

$$\begin{aligned}\tau(d) &:= \min\{k \in \{m_0, \dots, W\} : d \leq W(r) \text{ for every integer } r \in [k, W]\}, \\ \mathcal{D}(P) &:= \sum_{j: \text{copy}} \min\{\ell_j, \tau(d_j) - 1\}.\end{aligned}$$

Consider any valid parser which encodes each selected match by the cheaper of its token and its individual literals, retaining the selected conceptual boundary. Let F count its conceptual starts i inside reference copies $[p_j, q_j]$ where $r := q_j - i + 1 \geq \tau(d_j)$ but the chosen phrase has length less than r . Then

$$C_B^{\text{heur}}(x_1^n) - \Gamma_n \leq \frac{K_W}{W}n + K_W Z + b\mathcal{D}(P) + K_W F. \quad (21)$$

Exact greedy search has $F = 0$. Moreover, $\tau(d) \leq t_B$, so $\mathcal{D}(P) \leq (t_B - 1)Z$.

The condition $F = 0$ certifies the required reference-suffix-reaching property; the complete parse can still differ from exact greedy parsing.

Proof. The full-window tail ensures that $\tau(d)$ exists. The condition on every remaining length handles nonmonotone schedules. Inside a reference copy, its suffix of length $r \geq \tau(d)$ is a legal copy at the same distance, including when copies overlap. Exact greedy parsing therefore reaches the reference end.

For an arbitrary parser, every nonlast start in a reference copy either counts toward F or has fewer than $\tau(d)$ reference symbols remaining. A failed phrase has length less than $r \leq W$ and costs at most K_W ; it is necessarily nonlast. The other nonlast phrases stay within the final $\min\{\ell, \tau(d) - 1\}$ reference positions and consume disjoint intervals there. Their total cost is at most b times this number by the cheaper-representation rule. Charge at most K_W

for the last start if its phrase is shorter than W . A reference literal contains at most one start. Finally, all phrases of length at least W cost at most K_W/W per consumed symbol. A phrase crossing a reference boundary is charged only where it starts; reference phrases with no new start require no charge. Summing proves (21). $\square$

The next corollary turns this failure count into a finite search-budget bound for a specified index policy.

Corollary 5.4 (Capped multi-index search) *Fix index groups g with seed lengths and extension limits $1 \leq k_g \leq L_g \leq W$, histories $V_g \leq W$, bucket counts K_g , and integer inspection caps $S_g \geq 1$. Require that every supported $r \leq W$ has a queried group $g(r)$ satisfying*

$$k_{g(r)} \leq r \leq L_{g(r)}, \quad V_{g(r)} \geq W(r).$$

Each index retains every eligible input position, including positions inside previously encoded phrases. At each conceptual start, query all groups having a complete seed and inspect their buckets in most-recent-first order, stopping at S_g nodes or the bucket end. Verify the full seed and test candidates exactly through length L_g or the input end, considering every supported admissible length in $[k_g, L_g]$ found. A group with $L_g < W$ may stop earlier at its first verified admissible match of length L_g . With nearest-distance ties and an ordinary match cap W , a group with $L_g = W$ may likewise stop at its first verified admissible length- W match; otherwise it continues to the inspection cap or bucket end. Select a longest admissible match across the groups. Use the cheaper representation as in Proposition 5.3.

The dedicated short-length chains use $k_g = L_g = \ell$ and $V_g = W(\ell)$, so each stops at its first verified match. The largest class uses $k_g = t_B$, $L_g = W$, and $V_g = W$ to search longer matches. More general shared indexes satisfy the same coverage condition; a hit shorter than their maximum served length does not itself justify stopping. The full- W stop preserves nearest-longest selection: later nodes in that group are older and cannot yield a longer permitted match. Selection across groups still uses all their returned candidates. It does not apply to the unrestricted-length parser of Corollary 5.11.

Fix the input and reference parse P . Let $\mathcal{Q}(P)$ contain all positions i in reference copies $[p_j, q_j]$ with remaining suffix length $r_i := q_j - i + 1 \geq \tau(d_j)$. Fix a covering group $g_i := g(r_i)$ and let d_i^ be the nearest legal full-length witness distance:*

$$d_i^* := \min\{d : 1 \leq d \leq \min\{W(r_i), i - 1\}, \quad x_{i-d}^{i-d+r_i-1} = x_i^{i+r_i-1}\}.$$

The reference distance d_j is such a witness. Among the $d_i^ - 1$ closer positions, let A_i count seeds equal to the query's k_{g_i} -symbol seed. Define*

$$v_i := \begin{cases} 0, & S_{g_i} \geq d_i^*, \\ 1, & S_{g_i} \leq A_i, \\ \min\left\{1, \frac{d_i^* - 1 - A_i}{K_{g_i}(S_{g_i} - A_i)}\right\}, & \text{otherwise.} \end{cases}$$

Choose hashes independently of the input as in Theorem 4.7. Then $\mathbb{E}_{\text{hash}} F \leq \sum_{i \in \mathcal{Q}(P)} v_i$ and

$$\mathbb{E}_{\text{hash}} C_B^{\text{heur}}(x_1^n) - \Gamma_n \leq \frac{K_W}{W} n + K_W Z + b\mathcal{D}(P) + K_W \sum_{i \in \mathcal{Q}(P)} v_i. \quad (22)$$

The policy visits at most $n \sum_g S_g$ candidate nodes. History, lookahead, and indexes use $O(W + \sum_g (V_g + K_g))$ symbols/entries, with entry widths accounted for separately.

Proof. The covering group retains the witness and tests its required length. Let Z_i count closer unequal seeds colliding with the query hash. The witness has bucket rank $1 + A_i + Z_i$, and $\mathbb{E}_{\text{hash}} Z_i \leq (d_i^* - 1 - A_i)/K_{g_i}$. If the group stops earlier at a verified admissible L_{g_i} -match, that match already reaches at least r_i . Otherwise, a witness within the inspection budget is visited and exact verification reaches r_i . Thus, when $S_{g_i} > A_i$, a suffix-reaching failure requires the event $Z_i \geq S_{g_i} - A_i$. Markov's inequality gives the third branch of v_i ; the other branches follow from rank at most d_i^* and the trivial probability bound. No independence among collisions is needed. Every counted failure occurs in the fixed set $\mathcal{Q}(P)$, so summing over that set avoids independence assumptions at parser-selected starts. Apply Proposition 5.3. The visit and storage bounds follow from the caps and retained arrays. $\square$

If $\mathcal{G}_{\text{short}}$ consists of the dedicated fixed-length groups, their first-match rule sharpens the expected total visit bound to

$$n \sum_{g \in \mathcal{G}_{\text{short}}} \min\{S_g, 1 + V_g/K_g\} + n \sum_{g \notin \mathcal{G}_{\text{short}}} S_g.$$

This follows by summing the fixed-input work bound of Section 4.4 over all possible positions; the actual parse queries only a subset. Thus hash-dependent phrase starts require no independence assumption. The worst-case cap bound remains valid.

For comparison, with $I_g := \{1, \dots, n - k_g + 1\}$, $N_{g,i} := \min\{V_g, i - 1\}$, and $M_{g,i}$ the number of retained equal seeds, the bucket-size bound gives the coarser alternative

$$\mathbb{E}_{\text{hash}} F \leq U := \sum_g \sum_{i \in I_g} \min \left\{ 1, \frac{M_{g,i} + (N_{g,i} - M_{g,i})/K_g}{S_g + 1} \right\}.$$

Indeed, a failure requires overflow of a covering bucket; apply (15) at the integer threshold $S_g + 1$. The rank bound instead charges only candidates ahead of a successful witness. On a constant string it gives zero with cap one, even when the bucket-size bound is linear in n .

Outside E exceptional qualifying positions, suppose $d_i^* \leq D_{g_i}$ and $A_i \leq S_{g_i}/2$. Then

$$\frac{\mathbb{E}_{\text{hash}} F}{n} \leq \min \left\{ 1, \frac{E}{n} + \max_g \frac{2D_g}{K_g S_g} \right\}. \quad (23)$$

These are explicit recurrence and partial-seed conditions; polynomial substring complexity alone is not asserted to imply them. For one covering index with $V = K = W$, $S = \lceil W^{1/s} \rceil$, and fixed $s > 1$, the condition $A_i \leq S/2$ at every qualifying position makes the added rate $O(BW^{-1/s})$. Inputs also satisfying Corollary 5.8, with $W \leq n$, therefore satisfy $(\mathbb{E}_{\text{hash}} C_B^{\text{heur}} - \Gamma_n)/n = O(BW^{-1/s})$, with $S = o(V)$, $O(nW^{1/s})$ visits, and $O(W)$ entries. The expectation is for each finite input, not a pathwise assertion for one fixed choice of hashes.

A deterministic example needs no partial-seed condition. On a globally p -periodic input with $p \leq W$ and all covering caps at least p , the nearest witness has distance at most p : use the original reference distance d_j if $d_j < p$, and distance p otherwise. Hence $F = 0$ for every hash choice. Direct charging within periodic regions gives a stronger bound and also accommodates changes of pattern.

Corollary 5.5 (Piecewise-periodic inputs) *Use the policy and code-cost assumptions of Corollary 5.4, with $W \geq 2$. Suppose x_1^n is a concatenation of J nonempty intervals I_j , each with a period p_j satisfying $1 \leq p_j \leq \min\{|I_j|, p_*\}$, where $p_* \leq W$. Put $\Pi := \sum_{j=1}^J p_j$ and require every covering index cap to satisfy $S_g \geq p_*$. Then, for every hash choice,*

$$C_B^{\text{heur}}(x_1^n) - \Gamma_n \leq \frac{K_W}{W}n + b\Pi + J\{b(t_B - 1) + 2K_W\}. \quad (24)$$

In particular, bounded literal cost and $t_B = O(B)$ give payload cost $O(nB/W + \Pi + JB)$. The segmentation and periods are analytical certificates; the encoder and decoder do not need them.

Proof. Charge every conceptual phrase of length at least W at most K_W/W per consumed symbol, giving the first term. For an interval $I_j = [a_j, e_j]$, consider a selected start $i \geq a_j + p_j$ with at least t_B interval symbols remaining. Set $r := \min\{W, e_j - i + 1\}$. Periodicity supplies a length- r copy at distance p_j . Since $r \geq t_B$, its full window makes that distance admissible. A covering index retains the witness, whose bucket rank is at most $p_j \leq S_g$, independent of collisions. The search either visits this witness or stops earlier at a verified admissible match of length $L_g \geq r$. In either case it finds a match of length at least r . Consequently, any selected phrase shorter than W at such a start reaches the interval end.

Charge each remaining short phrase to the interval containing its start. Those contained in the first p_j positions together cost at most bp_j by literal fallback. At most one short phrase starts in that prefix and crosses its end; charge it K_W . After the prefix, any short phrase not reaching the interval end must start within its final $t_B - 1$ positions. Such contained phrases cost at most $b(t_B - 1)$ in total, and at most one further short phrase reaches or crosses the interval end, costing at most K_W . Overlapping prefix and terminal regions only increase this upper bound; a phrase entering from a preceding interval only removes starts. Summing proves (24). $\square$

For one globally p -periodic input, this yields

$$\frac{C_B^{\text{heur}} - \Gamma_n}{n} = O\left(\frac{B}{W} + \frac{p + B}{n}\right).$$

The period is charged once, so the large-file payload rate at fixed W is $O(B/W)$. With W comparable to n , the piecewise bound is $O(\Pi + (J + 1) \log n)$ bits, apart from headers. One seed-length-one full-history index with $S = p_*$ uses at most np_* visits and $O(W)$ entries, even with one bucket. Thus $p_* = o(W)$ permits a sublinear inspection cap. For a fixed cap, the same output satisfies the bound for every qualifying segmentation, so one may minimize the analytical bound over them without transmitting one. The argument uses ordinary copies and supports lengths through W ; the logarithmically capped run format of Section 6.3 has different length support.

These bounds count candidate visits. Symbolwise extension can require $O(W)$ work per candidate; hashing and index maintenance also cost time. A single seed $k = t_B$ covers only suffixes of length at least t_B ; the finer $\mathcal{D}(P)$ requires the coverage specified above.

Theorem 5.6 (Complete periodic Huffman coding under capped search) *Use the policy of Corollary 5.5 on a globally p -periodic input, with $p \leq W$ and ordinary match*

cap W . Put $S_{\max} := \max_g S_g$ and $D_* := \min\{W, pS_{\max}\}$. For sufficiently large W that $K_W < bW$, fit the four codes of Proposition 4.5 to the emitted parse, retaining its original literal fallback, and transmit the tables and framing of (12). Then

$$\limsup_{n \rightarrow \infty} \frac{C_H(x_1^n)}{n} \leq \frac{3 + \lceil \log D_* \rceil}{W}. \quad (25)$$

If ties among visited longest matches are resolved by smallest distance, the sharper complete-code bound is

$$\limsup_{n \rightarrow \infty} \frac{C_H(x_1^n)}{n} \leq \frac{3}{W}, \quad (26)$$

without an upper bound on the inspection caps. With this tie rule, $B = \log W$, and $t_B = O(B)$, the finite-input bound for $n \geq W$ is

$$C_H(x_1^n) \leq \frac{3n}{W} + O((p + B)B + \log(n + 1)), \quad (27)$$

with fixed alphabet and code-cost constants.

Proof. Every visited candidate has distance at most D_* . To see this, fix a queried group g . If its available history is shorter than pS_g , all candidates have smaller distance. Otherwise the S_g positions at distances $p, 2p, \dots, S_gp$ have the same seed as the query, including overlapping seeds. With complete retention and most-recent-first traversal, the first S_g bucket entries lie within that range, regardless of collisions; early stopping only shortens this prefix. Distances also never exceed W . Groups without a complete query seed contribute no candidates.

The proof of Corollary 5.5 bounds the number of conceptual phrases shorter than W by $Q := p + t_B + 1$, and the number of emitted literals by

$$N \leq p + t_B - 1 + 2K_W/b.$$

The condition $K_W < bW$ prevents fallback on a full-length phrase. Thus all but finitely many matches have length W , all but finitely many literal counts are zero, and $M/n \rightarrow 1/W$. For each of these two fields, give the dominant value a one-bit word and encode the finitely many exceptional values under the other branch. Their combined cost is at most $2M + O_W(1)$. A fixed code for offsets costs at most $M \max\{1, \lceil \log D_* \rceil\}$, and literals cost $O_W(1)$. All four supports and their labels are bounded at fixed W , uniformly in n . Hence (12) charges $O_W(1) + O(\log(n + 1))$ bits. Huffman optimality and division by n prove (25) for the complete code.

For nearest ties, let p_0 be the primitive period of the infinite extension of the period word, so $p_0 \mid p$. A full- W equality, with $W \geq p$, covers every residue modulo p ; its shift therefore preserves the cyclic period word and is a multiple of p_0 . Whenever a full- W match can occur, distance p_0 is available. A group covering length W has $L_g = W$, and the witness has bucket rank at most $p_0 \leq p \leq S_g$. No closer candidate is a full- W match, since its distance would be a smaller positive multiple of p_0 . Thus the witness is visited even with first-full-match stopping. Nearest ties select p_0 for every full- W match. The offset field now also has one dominant symbol and finitely many exceptions. Giving each dominant symbol one bit proves (26).

For the finite bound, $Q, N = O(p + B)$ and $M \leq n/W + Q$. Each of the three integer fields has at most Q exceptional occurrences outside its dominant symbol. Prefixing an index for each exceptional value gives total integer payload $3M + O(Q \log(Q + 1))$. Literal payload is $O(N)$. The tree labels cost $O((Q + 1)B)$, since $p \leq W$ and all literal counts are at most $N = O(p + B)$. Adding framing and using $\log(Q + 1) = O(B)$ proves (27). $\square$

No period or special periodic-pattern token is supplied to the decoder. With nearest-distance ties, compare codes for the same parse. For $W = 2^B$, let C_{fix} encode that parse using its original fixed-width distance fields and match cost $B + a(W)$ for a length- W copy, where $0 \leq a(W) = O(B)$; assume complete header cost $o(n)$ at fixed W . The proof gives $M_W/n \rightarrow 1/W$ for the number M_W of length- W copies, with only $O_W(1)$ other tokens. Consequently,

$$\lim_{n \rightarrow \infty} \frac{C_{\text{fix}}(x_1^n)}{n} = \frac{B + a(W)}{W} = \Theta(B/W), \quad \limsup_{n \rightarrow \infty} \frac{C_{\text{H}}(x_1^n)}{n} \leq \frac{3}{W}.$$

For a fixed periodic source, this is a logarithmic-factor improvement as $W \rightarrow \infty$ after the large-file limit. It compares fixed-width and Huffman representations of the specified ordinary-copy parse; the historical 1977 triple format is compared separately in Theorem 6.2. The gain requires bounded extra-bit cost on the dominant values: a growing raw extra-bit field can retain a logarithmic cost even when its range code is predictable.

With the optional full- W stop, each full-length covering group inspects at most p_0 nodes at a start admitting a full- W match; on a constant input this is one visit. For bounded p , the finite remainder is $O(B^2 + \log(n + 1))$; this remainder must still be charged when W grows with n . The iterated rate amortizes one set of fitted tables over the whole input. Refreshing tables in successive blocks incurs their costs repeatedly. Without nearest ties, a one-bucket index with cap W on a constant input can cycle through offsets $1, \dots, W$, retaining offset entropy $\log W$ and rate of order $(\log W)/W$. Thus the tie rule controls coding entropy as well as match selection.

The bound $3/W$ concerns symbolwise Huffman coding, not a limitation of the parse. For $n \geq W$, under nearest ties, the integer streams have default values $0, W, p_0$ and at most Q exceptions each. Standard enumerative coding [44] describes a length- m stream over U labels with s exceptions using

$$O(\log U + \log(s + 1) + 1) + \left\lceil \log \binom{m}{s} \right\rceil + s \lceil \log \max\{1, U - 1\} \rceil$$

bits: send the default and exception count, rank the exceptional positions, and send their values. For an empty stream or singleton alphabet, the corresponding empty fields cost zero. Send n, N, M, W self-delimitingly and the N literals at fixed width; the integer universes are $N + 1, \min\{n, W\}$, and W . At fixed W, p , the universes and exception counts are bounded, so the complete length is $O_{W,p}(\log(n + 1))$. All defaults, including p_0 , are charged. This changes only the representation of the same parse and supplies no period to the decoder for free.

For a finite word y , let

$$\delta(y) := \max_{1 \leq m \leq |y|} \frac{D_m(y)}{m}.$$

The established LZ77 bound

$$z(y) = O\left(\delta(y) \log\left(1 + \frac{|y|}{\delta(y)}\right)\right) \quad (28)$$

follows from substring-complexity counting; it is a consequence of the stronger alphabet-dependent bound of Kociumaka, Navarro, and Prezza [35, Lemma II.4], based on Raskhodnikova et al. [34]. The added 1 in the logarithm handles the endpoint where $\delta(y)$ is comparable to $|y|$. Equation (28) is an existing result; the transfer in Theorem 5.2 is the step needed for the length-dependent-window model.

Corollary 5.7 (Polynomial substring complexity) *Suppose $D_m(x_1^n) \leq Am^s$ for every $n \geq m$, with fixed $A \geq 1$ and $s \geq 1$. If $t_B = O(B)$, the exact greedy family satisfies*

$$\mathcal{R}_B(x) \leq \begin{cases} O(BW^{-1/s}), & s > 1, \\ O(B^2/W), & s = 1. \end{cases} \quad (29)$$

The constants may depend on A, s and the code-cost bounds.

Proof. For $s = 1$, every reference block y has $\delta(y) \leq A$, so (28) gives $z(y) = O(\log(1 + |y|))$. For fixed $s > 1$, use the distinct-substring argument in the proof of Claim 4 of [34]. In a full length- W reference block, let n_k count ordinary greedy phrases of length k , excluding the final phrase. For $1 \leq u \leq W/2$,

$$\sum_{k \leq u} kn_k \leq D_{2u} + 2u \leq A(2u)^s + 2u.$$

Thus a dyadic length class $u/2 < k \leq u$ contains $O(Au^{s-1} + 1)$ phrases. Summing the geometric series up to u gives $O(Au^{s-1} + \log u)$ phrases of length at most u . There are at most W/u longer phrases, plus the final phrase. Take a power of two u within a factor two of $W^{1/s}$; for sufficiently large W it is below $W/2$. Since $s > 1$ is fixed, $\log u = O(u^{s-1})$, and $z(y) = O(W^{1-1/s})$.

There are $\lfloor n/W \rfloor$ full reference blocks, while the final block contributes at most W phrases. Divide (20) by n and let $n \rightarrow \infty$ at fixed W . Since $K_W + b(t_B - 1) = O(B)$, the two reference counts give $O(BW^{-1/s})$ for $s > 1$ and $O(B \log W/W) = O(B^2/W)$ for $s = 1$. $\square$

The exponent $1/s$ improves the elementary $1/(s+1)$, and for $s > 1$ the dyadic argument also removes a logarithmic factor. Ordinary single-window LZ77 satisfies these bounds as well. The finite-input orders have matching converses in Theorem 5.9. Polynomial rates for particular zero-entropy sources also have precedent [45].

5.2 Finite-input bounds and minimax preservation

The preceding rate takes the input-length limit before the window limit. Its proof also yields a uniform finite-input statement when the last reference block is charged by its substring complexity rather than by its raw length.

Corollary 5.8 (Finite-input complexity bound) *Use the exact greedy family of Theorem 5.2, with $t_B = O(B)$, $W = 2^B \leq n$, and supported lengths through at least W . Suppose $D_m(x_1^n) \leq Am^s$ for all $1 \leq m \leq n$, with fixed $A \geq 1$ and $s \geq 1$. Uniformly for sufficiently large $W \leq n$,*

$$\frac{C_B(x_1^n) - \Gamma_n}{n} \leq \begin{cases} O(BW^{-1/s}), & s > 1, \\ O(B \log(1+W)/W), & s = 1. \end{cases} \quad (30)$$

Consequently, choosing the largest power of two at most n gives $C_B/n \leq \Gamma_n/n + O((\log n)n^{-1/s})$ for $s > 1$, and $C_B/n \leq \Gamma_n/n + O((\log n)^2/n)$ for $s = 1$.

Proof. Write $n = aW + r$, $0 \leq r < W$. For $s > 1$, the dyadic argument in Corollary 5.7 bounds the ordinary phrase count of every length- m reference block by $cm^{1-1/s}$. Every such block inherits the assumed substring bound. Hence

$$Z_W \leq c\{aW^{1-1/s} + \mathbf{1}_{\{r>0\}}r^{1-1/s}\}.$$

Substitution in (20), whose coefficients are $O(B)$, gives $O(BW^{-1/s} + BW^{1-1/s}/n)$. For $s = 1$, use $z(y) \leq c \log(1 + |y|)$ from (28) to obtain $O(B \log(1+W)/W + B \log(1+W)/n)$. In each case the second term is bounded by the first because $W \leq n$. $\square$

This family begins with an empty history and starts compressing immediately. No raw prefix of length W is required. Within this bound, the largest allowed window gives the strongest eventual guarantee; there is no intrinsic intermediate optimum. The constants are uniform in n, W but depend on A, s and the code model. Header cost Γ_n must be controlled along the chosen $W(n)$: an $o(n)$ statement for each fixed W alone is insufficient.

The finite-input orders have a matching information-theoretic converse. The following statement includes complete code lengths.

Theorem 5.9 (Finite-input minimax order) *Fix $A \geq 2$ and $s \geq 1$, independently of n , and let*

$$\mathcal{F}_{n,s,A} := \{x \in \{0, 1\}^n : D_m(x) \leq Am^s \text{ for all } 1 \leq m \leq n\}.$$

Let C range over injective maps from $\{0, 1\}^n$ to finite binary strings whose images are prefix-free, with n known to the decoder. Then, as $n \rightarrow \infty$,

$$L_{n,s,A}^* := \inf_C \max_{x \in \mathcal{F}_{n,s,A}} |C(x)| = \begin{cases} \Theta(n^{1-1/s} \log n), & s > 1, \\ \Theta((\log n)^2), & s = 1. \end{cases} \quad (31)$$

The constants may depend on A, s . A single exact greedy WLZ format, without knowledge of A or s , attains both upper orders.

Proof. For the upper bound, take $n \geq 4$, $B = \lfloor \log n \rfloor$, and $W = 2^B$. Use minimum match length two and the decoder-known schedule

$$W_B(2) := 1, \quad W_B(\ell) := W \quad (\ell \geq 3).$$

Support all feasible lengths and overlapping copies. Parse by exact longest admissible matching, with a fixed tie rule. Encode a binary literal a as $0a$. Encode a match by a 1 flag, the

Elias gamma code of $\ell - 1$, and $d - 1$ in exactly $\log W_B(\ell)$ bits. The gamma code reveals the distance-field width; length-two matches have distance one and no distance bits. Literal cost is $b = 2$, and match cost is at most

$$\log W_B(\ell) + 2 + 2\log(\ell + 1).$$

Replace an expensive conceptual match by its literals when cheaper, retaining its conceptual boundary. Decoding stops after n symbols, so valid complete encodings form a prefix code. The schedule and B are determined by n ; no schedule table or fitted model is transmitted. Thus headers can be constant for shared n , or $O(\log n)$ after encoding n self-delimitingly. Small n can be encoded literally. This format has $t_B = 3 = O(B)$ and uniform logarithmic length costs. Corollary 5.8, with $n/2 < W \leq n$, therefore gives both complete-code upper bounds.

For the lower bound when $s > 1$, put

$$d = \lceil n^{1/s} \rceil, \quad k = \lfloor n/(2d) \rfloor.$$

Partition the first $2dk$ positions into k blocks of length $2d$. In each block independently place one 1 in one of its first d positions, and fill all other positions with zero. There are d^k distinct strings. Consecutive ones have distance at least $d + 1$, so every length- m substring with $m < d$ contains at most one 1. Hence

$$D_m \leq m + 1 \leq 2m^s \quad (m < d), \quad D_m \leq n \leq d^s \leq m^s \quad (m \geq d).$$

The constructed family lies in $\mathcal{F}_{n,s,2} \subseteq \mathcal{F}_{n,s,A}$. Kraft's inequality forces every candidate code to have length at least $\log(d^k)$ on some family member. For fixed $s > 1$,

$$k \log d \sim \frac{1}{2s} n^{1-1/s} \log n.$$

For $s = 1$, use the binary prefix families of Kociumaka, Navarro, and Prezza [35, Definition IV.2, Lemma IV.3, and the binary construction in the proof of Lemma IV.4]. Their members satisfy $D_m \leq 2m$, while their logarithmic cardinality is $\Omega((\log n)^2)$. They therefore lie in $\mathcal{F}_{n,1,A}$, and the same prefix-code counting argument proves the lower bound. This linear-complexity converse is an existing result. $\square$

Ordinary LZ77 can attain these orders as well. The theorem establishes optimal worst-case finite-input orders when the window is comparable to the input length; its hard strings may depend on n .

Corollary 5.10 (Preservation across window schedules) *Fix $A \geq 2$ and $s \geq 1$. Consider a collection of complete prefix-coded exact greedy WLZ families satisfying Theorem 5.2, with common bounds on literal and logarithmic length costs, $t_B \leq c_t B$, and $cn \leq W = 2^B \leq n$ for common $c_t, c > 0$. Schedules are shared or transmitted, and all header costs are $O((\log n)^2)$, uniformly over inputs and the collection. Every such family satisfies*

$$\max_{x \in \mathcal{F}_{n,s,A}} C_{W,n}(x) = \Theta(L_{n,s,A}^*),$$

with uniform constants over the collection and the orders in (31).

Proof. Corollary 5.8 and the header bound give the upper orders. Each complete encoder is among the prefix codes in Theorem 5.9, so its converse gives the lower orders. $\square$

For example, the literal/gamma format in the theorem permits arbitrary integer widths $w_B(\ell) \in \{0, \dots, B\}$ below a threshold $t_B \geq 2$, for $2 \leq \ell < t_B$, and width B thereafter. For $t_B = O(B)$, a self-delimiting threshold followed by its short-width table costs $O(B \log(B+1))$ bits. Thus substantial short-match restrictions, including nonmonotone tables, preserve the optimal orders even when their descriptions are transmitted. The length-two schedule is the simplest attaining example.

The same parsing comparison also yields a stronger conditional payload bound when the fitted entropy tables are supplied without charge, as in Proposition 4.5. This comparison concerns the representation of the selected conceptual matches before the old fixed-cost literal expansion.

Corollary 5.11 (Linear-complexity Huffman payload) *Use the exact greedy schedule from Theorem 5.9, with $W = 2^{\lceil \log n \rceil}$, $W_B(2) = 1$, and $W_B(\ell) = W$ for $\ell \geq 3$. Encode every selected conceptual match as a copy and every unmatched symbol as a literal, using the four Huffman streams. For binary inputs satisfying $D_m(x_1^n) \leq Am$ for every $1 \leq m \leq n$, with fixed $A \geq 1$, the payload obeys*

$$C_H^{\text{pay}}(x_1^n) = O(\log n \log \log n). \quad (32)$$

Adding $O(\log n)$ stream framing preserves this order.

Proof. The start-counting argument in Theorem 5.2 gives $P \leq t_B Z_W$ for the conceptual phrase count: at most $t_B - 1$ nonlast starts lie in the final $t_B - 1$ sites of each reference copy, and at most one further start is last. A reference literal contains at most one start. This counts long and crossing phrases as well. Here $t_B = 3$, and the at most two reference blocks have total phrase count $Z_W = O(\log n)$ by (28). Thus $P \leq 3Z_W = O(\log n)$, and (11) proves (32). $\square$

Match selection and conceptual boundaries are unchanged; only their representation is reconsidered under the fitted field codes. The result improves the fixed-field logarithmic factor for this schedule, not the polynomial exponent. For fixed $s > 1$, under $D_m(x_1^n) \leq Am^s$ for every $1 \leq m \leq n$ with fixed A , the same reasoning gives $P = O(n^{1-1/s})$ and retains the order $O(n^{1-1/s} \log n)$. An arbitrary threshold $t_B = O(B)$ can introduce an additional factor in $P \leq t_B Z_W$, so the refinement is not uniform over all schedules in Corollary 5.10.

The table contribution can be quantified using the existing converse. Let X be uniform on the binary lower-bound family in the $s = 1$ proof of Theorem 5.9, so $H(X) = \Omega((\log n)^2)$ and $D_m \leq 2m$ [35, Definition IV.2 and Lemmas IV.3–IV.4]. With deterministic parsing and Huffman tie conventions, let $T := T(X)$ be the labeled tables, and let $Q(X)$ be the payload with its $O(\log n)$ framing. For each fixed T , these framed payloads form a conditional prefix code and recover X . Therefore

$$H(T) = H(X) - H(X | T) \geq H(X) - \mathbb{E}|Q(X)| = \Omega((\log n)^2). \quad (33)$$

Any standalone prefix description of the tables has at least this expected length on that family. Supplying the $O(\log n)$ -bit framing first does not change the lower order, since

conditioning on it removes at most $O(\log n)$ entropy. This is an average, and hence some-input, lower bound, not a claim about every input.

Conversely, $P = O(\log n)$ bounds the support sizes, and every integer label needs $O(\log n)$ bits. Equation (12) therefore gives $O((\log n)^2)$ complete length. For fixed $A \geq 2$, the complete four-field format attains worst-case order $\Theta((\log n)^2)$ on $\mathcal{F}_{n,1,A}$: its smaller conditional payload locates distinguishing information in the labeled model while preserving the complete minimax order.

5.3 Exact-greedy universality and its limitations

The suffix-reaching argument also transfers classical stationary-ergodic LZ universality [31] to the restricted WLZ dictionary. Charging failed reference blocks by their literal lengths avoids requiring a quantitative recurrence tail at greedy-selected starts.

Lemma 5.12 (Charging matched and failed blocks) *Retain the exact greedy model and uniform code-cost bounds above, and put $A_0 := \max\{b, a_0\}$. Partition x_1^n into J nonempty consecutive blocks of lengths at most m , where $t_B \leq m \leq W$. Choose as good any collection of full length- m blocks having a backward match within distance W whose source starts in the input; mark all remaining blocks bad. Let J_g count good blocks and V count symbols in bad blocks. Then*

$$C_B^{\text{greedy}}(x_1^n) - \Gamma_n \leq J \left[B + A_0 + a_1 \log \left(1 + \frac{n}{J} \right) \right] + b(t_B - 1)J_g + bV. \quad (34)$$

An ordinary match cap of W is sufficient.

Proof. Assign each conceptual phrase to the block containing its start and distinguish the last start in each block that contains one. In a good block, any start with at least t_B symbols remaining has a legal copy of the entire remaining suffix at the witness distance. Greedy parsing therefore reaches the block's end. Every nonlast start lies in the final $t_B - 1$ symbols, and its phrase ends before another start in that block. These disjoint intervals cost at most $b(t_B - 1)$ in total by literal fallback. In a bad block, the nonlast intervals similarly remain inside the block and cost at most its length times b .

The distinguished last-start phrases are disjoint even when they cross block boundaries. There are at most J of them, and their lengths sum to at most n . Each costs at most $B + A_0 + a_1 \log(\ell + 1)$, including a literal. Pad their length list with zeros to J entries and apply concavity of $\log(1 + u)$; their total cost is at most the first term in (34). Summing the charges proves the result. The reference boundaries do not reset the encoder. $\square$

Theorem 5.13 (Universality of exact greedy WLZ) *Suppose $t_B = o(B)$, every length from t_B through $W = 2^B$ has the full window, and the exact greedy family satisfies the preceding uniform cost bounds. Assume complete prefix coding with $\Gamma_n = o(n)$ for each fixed B . For every finite-alphabet stationary ergodic source of entropy rate $h \geq 0$, put $\bar{K}_m := B + A_0 + a_1 \log(m + 1)$. For fixed $t_B \leq m \leq W$,*

$$\limsup_{n \rightarrow \infty} \frac{C_B^{\text{greedy}}(X_1^n)}{n} \leq \frac{\bar{K}_m + b(t_B - 1)}{m} + b \Pr\{R_m > W\} \quad \text{almost surely,} \quad (35)$$

where R_m is the backward recurrence distance in a two-sided stationary extension. Consequently,

$$\lim_{B \rightarrow \infty} \limsup_{n \rightarrow \infty} \frac{C_B^{\text{greedy}}(X_1^n)}{n} = h \quad \text{almost surely.} \quad (36)$$

Both conclusions hold with ordinary match cap W .

Proof. For each of the m phases, partition the input into length- m blocks with shorter end blocks. Mark the end blocks and full blocks starting at positions at most W bad, and designate every other block good exactly when $R_m \leq W$. Across all phases each interior position occurs once as a block start. Hence the average bad-symbol fraction is at most

$$\frac{1}{n} \sum_{i=W+1}^{n-m+1} \mathbf{1}\{R_m(T^{i-1}X) > W\} + O((W+m)/n),$$

where T is the one-symbol shift. Ordinary ergodicity bounds the upper limit of this average by $\Pr\{R_m > W\}$; no ergodicity under the m -symbol shift is needed. Apply Lemma 5.12 to a phase minimizing the bad-symbol count. For every phase, $J/n \rightarrow 1/m$, $n/J \rightarrow m$, and $J_g \leq J$, proving (35).

The recurrence theorem gives $\log R_m/m \rightarrow h$ almost surely [32, 45]. For any $\epsilon > 0$, choose $m_B := \lfloor B/(h + \epsilon) \rfloor$, which eventually lies between t_B and W . The recurrence theorem and bounded convergence give $\Pr\{R_{m_B} > 2^B\} \rightarrow 0$. Also $\bar{K}_{m_B}/m_B \rightarrow h + \epsilon$ and $t_B/m_B \rightarrow 0$. Thus the outer upper limit in (36) is at most $h + \epsilon$, including when $h = 0$.

For the converse, write $I_n := -\log P_n(X_1^n)$. For every fixed B , Kraft's inequality gives $\Pr\{C_B^{\text{greedy}}(X_1^n) < I_n - 2 \log n\} \leq n^{-2}$. Borel–Cantelli and the Shannon–McMillan–Breiman theorem yield $\liminf_n C_B^{\text{greedy}}(X_1^n)/n \geq h$ almost surely. Intersecting the probability-one events over the countably many B, m and positive rational ϵ , then letting $\epsilon \downarrow 0$, proves (36). $\square$

Minimum-bit WLZ in the same format inherits universality by comparison:

$$\lim_{B \rightarrow \infty} \limsup_{n \rightarrow \infty} \frac{C_B^*(X_1^n)}{n} = h \quad \text{almost surely.} \quad (37)$$

For this ideal parser, matched m -blocks and literal-coded failures give the slightly sharper phase-averaged bound

$$\limsup_{n \rightarrow \infty} \frac{C_B^*(X_1^n)}{n} \leq \frac{B + a(m)}{m} + b \Pr\{R_m > W\}. \quad (38)$$

The greedy result requires neither unbounded match lengths nor a rate for the vanishing recurrence probability.

Finally, a fixed number of short-match classes does not establish a larger positive-entropy exponent. Appendix A proves an obstruction for exact greedy parsing with fixed-width long pointers and separately self-delimiting lengths. That model has expected redundancy at least a constant times $\log B/B$ on a uniform iid source. The result deliberately does not cover adaptive Huffman length coding or an entropy-cost raw-block fallback. Earlier SWLZ lower-bound analyses likewise depend on the precise coding model [46]. Wyner–Wyner's modified LZ construction already attains $O(1/B)$ under finite-memory assumptions [37]; removing a $\log B$ factor is distinct from proving a power B^{-p} with $p > 1$.

6 Strict Asymptotic Improvements

This section compares actual code lengths on the same source. The first two comparisons use separately specified encoder families: a growing match cap and a short codeword for that cap in the first comparison, and a calibrated window schedule with minimum-bit parsing or explicit parse transformation in the second. The polynomial-complexity bounds of the previous section do not supply these strict inequalities.

6.1 An asymptotic separation from the original fixed-field format

The original 1977 construction copies a prefix and appends one source symbol, using fixed-width pointer and phrase-length fields [5, p. 338]. We compare that format with a capped WLZ family whose maximum-length match has a cheap token codeword. This changes the length-code redundancy as well as the distance windows. The cap below is $\Theta(\log W)$, a different parameter regime from the growing-length family used for the polynomial-complexity bounds.

Lemma 6.1 (Uniform probability of finding a word) *In a uniform iid source on $q \geq 2$ symbols, the probability that a specified word of length $L \geq 1$ does not occur in a block of W symbols is at most*

$$\epsilon(W, L) := 2^{-\lfloor W/(8q^L) \rfloor} \leq 2 \exp\{-(\ln 2)Wq^{-L}/8\}. \quad (39)$$

The bound holds uniformly over the specified word.

Proof. First take a block of $M = 8q^L$ symbols, and let Z count occurrences of the word at its $N := M - L + 1$ possible starting positions. Its mean is $\mu := Nq^{-L} \geq 7$. Indicators at distance at least L are independent. At distance $1 \leq d < L$, their joint probability is either zero or $q^{-(L+d)}$, according to whether the word permits that overlap. Therefore

$$\text{Var } Z \leq \mu + 2Nq^{-L} \sum_{d=1}^{L-1} q^{-d} \leq \mu \left(1 + \frac{2}{q-1}\right) \leq 3\mu.$$

Chebyshev's inequality gives $\Pr\{Z = 0\} \leq 3/\mu \leq 3/7 < 1/2$. Partition the W symbols into $\lfloor W/M \rfloor$ independent blocks of size M . Absence from the whole window implies absence from every block, proving the first bound; the second follows by rounding. $\square$

Theorem 6.2 (Smaller-order redundancy at a common phrase cap) *Let the source be uniform iid on $q = 2^h$ symbols, where $h \geq 1$ is an integer. Let $W = 2^B$, with B a multiple of h , and give both encoders the same history size W and maximum phrase length*

$$L_B^{\text{cap}} := \left\lfloor \frac{B - 2 \log \log B}{h} \right\rfloor. \quad (40)$$

The baseline uses the original fixed-field triple format. WLZ uses exact greedy parsing, supports all lengths from a fixed threshold t through L_B^{cap} in the full window, and may use smaller windows below t . A WLZ cap token costs exactly $B + k_0$ bits for a constant k_0 , and all other

token costs lie between zero and $K_B = O(B + \log L_B^{\text{cap}})$. Initialization and header costs are $o(n)$ for fixed B . Then almost surely, for either the lower or upper large-file rate,

$$\mathcal{R}_{77}(B) = h + h \frac{\log B}{B} + 2h \frac{\log \log B}{B} + O(B^{-1}), \quad (41)$$

$$\mathcal{R}_W(B) = h + 2h \frac{\log \log B}{B} + O(B^{-1}). \quad (42)$$

In particular, the comparison holds directly on the same input:

$$\liminf_{n \rightarrow \infty} \frac{C_{77,B}(X_1^n) - C_{W,B}(X_1^n)}{n} \geq h \frac{\log B}{B} - O(B^{-1}). \quad (43)$$

Proof. Write $L := L_B^{\text{cap}}$, and call a source position bad if its next L symbols have no copy wholly contained in the preceding W symbols. Conditional on these next symbols, the past is independent and uniform. Lemma 6.1 bounds the bad-position probability by $\epsilon_B := \epsilon(W, L)$. The ergodic theorem bounds its almost-sure density by ϵ_B . This statement holds simultaneously for the countably many permitted B . Counting all bad source positions bounds the number of bad selected phrase starts, so no independence of greedy phrase boundaries is required.

At a nonbad start away from the file boundary, WLZ emits a cap token. The original encoder also emits a cap phrase, since it only needs to copy its first $L - 1$ symbols. The original codeword costs exactly

$$F_B := B + h \lceil \log_q L \rceil + h = B + \log L + O(1)$$

bits. This retains the paper's radix- q fields; for $q = 2$ the cost is $B + \lceil \log L \rceil + 1$. If N_{bad} counts bad source positions, each parser has at most $n/L + N_{\text{bad}} + O(1)$ phrases. The original has at least $n/L - O(1)$ phrases, and the number of WLZ cap tokens is between $n/L - N_{\text{bad}} - O(1)$ and n/L . Endpoint and initialization terms vanish for fixed B . Thus almost surely

$$\begin{aligned} \frac{F_B}{L} &\leq \liminf_n \frac{C_{77,B}}{n} \leq \limsup_n \frac{C_{77,B}}{n} \leq \frac{F_B}{L} + F_B \epsilon_B, \\ \frac{B + k_0}{L} - (B + k_0) \epsilon_B &\leq \liminf_n \frac{C_{W,B}}{n} \leq \limsup_n \frac{C_{W,B}}{n} \leq \frac{B + k_0}{L} + K_B \epsilon_B. \end{aligned}$$

Now $Wq^{-L} \geq (\log B)^2$, so $B\epsilon_B$ decays faster than every inverse power of B . Subtracting the WLZ upper bound from the baseline lower bound, and expanding L , proves all the assertions. $\square$

The coding condition has an explicit prefix-code realization. Give the cap match token the codeword 0. Encode each other literal or match-length symbol by 1 followed by a fixed-width index over at most $q + L_B^{\text{cap}}$ alternatives, and append the appropriate distance field to a match. Then $k_0 = 1$ and $K_B \leq B + 1 + \lceil \log(q + L_B^{\text{cap}}) \rceil$. Constant format tags only change k_0 . A Huffman code over the same complete token alphabet has no greater total token-code length than this code. With $o(n)$ model overhead and distance cost at most B bits per match, it inherits the WLZ upper bound and the strict gain (43). The equality (42) applies to the specified exact token costs; a better entropy coder may compress further.

In particular, the cap symbol must have no length-extra field whose cost grows with B . DEFLATE already assigns its maximum length 258 a distinct symbol with zero extra length bits [6, Section 3.2.5]; the theorem requires a growing cap and the stated code-cost bound, not the unchanged DEFLATE limits or an arbitrary choice of Huffman tables.

One can also compare with the original format after it chooses a different cap. This comparison also holds almost surely, like the common-cap statement above.

Proposition 6.3 (Original-format lower bound for every cap) *Under the same iid source and history size, let the original fixed-field triple encoder choose any finite phrase cap $J_B \geq 1$. Use exact full-window greedy copying followed by its mandatory final symbol, with no entropy-coded raw-block fallback. Uniformly over these cap choices, almost surely*

$$\liminf_{n \rightarrow \infty} \frac{C_{77,B,J_B}(X_1^n)}{n} \geq h + h \frac{\log B}{B} - O(B^{-1}), \quad (44)$$

and the same bound holds for $\liminf_n \mathbb{E} C_{77,B,J_B}(X_1^n)/n$. Consequently the WLZ rate in (42), which also holds in expectation, has smaller-order redundancy than this original format, almost surely and in expectation, even when the baseline chooses its own cap.

Proof. At an original phrase start the unrevealed input is fresh iid: the previous phrase consumed its first mismatching symbol, or stopped at its cap. More formally, the position immediately before each phrase start is a stopping time for the natural prefix filtration. Conditional on that prefix, a copy of length r at any specified distance has probability q^{-r} , including overlapping copies. A union bound over the W distances, followed by summation of tails, bounds the conditional mean phrase length by

$$\mu_B := B/h + 1 + (q - 1)^{-1}.$$

It is also bounded by J_B ; put $m_B := \min\{J_B, \mu_B\}$. Applying these conditional bounds to successive phrases whose starts lie in a finite file gives $\mathbb{E} N \geq (n - O(W))/m_B$, up to a terminal term that vanishes for fixed B, J_B . This uses conditional expectations, not independence of phrase lengths.

For the almost-sure bound, parse the infinite sequence X_1^∞ in the same way. Let L_j be the length of its j th phrase and $\mathcal{F}_j$ the σ -field of the input before that phrase. Then $1 \leq L_j \leq J_B$ and $\mathbb{E}[L_j \mid \mathcal{F}_j] \leq m_B$, so the partial sums of $L_j - \mathbb{E}[L_j \mid \mathcal{F}_j]$ form a martingale with increments bounded by J_B . The Azuma–Hoeffding inequality and the Borel–Cantelli lemma give $\limsup_k k^{-1} \sum_{j \leq k} L_j \leq m_B$ almost surely. Each phrase decision depends only on the history and the next J_B symbols, so the parse of X_1^n agrees with this parse in its k_n phrases that start at or before $n - J_B$, and these cover at least $n - J_B$ symbols. Hence $\liminf_n N/n \geq \liminf_n k_n/n \geq 1/m_B$ almost surely.

Each phrase costs $F_{B,J_B} := B + h[\log_q J_B] + h$ bits. The large-file rate is therefore at least $F_{B,J_B}/m_B$, almost surely and in expectation. If $J_B < B/(2h)$ this is greater than $2h$. Otherwise $F_{B,J_B} \geq B + \log B - O(1)$, and division by μ_B gives (44), with constants independent of J_B . The WLZ expectation bound follows directly by counting bad positions in expectation in the preceding proof. $\square$

In terms of W , the proved WLZ upper redundancy is $O(\log \log \log W / \log W)$, compared with the original format’s $\Omega(\log \log W / \log W)$ on this source. This is a strict asymptotic

separation caused by the length-code representation. The smaller windows alone do not cause this gain, and a modern single-window LZ77 encoder with the same cap-aware token code can obtain it too. Earlier modified-LZ constructions attain $O(1/B)$ redundancy under finite-memory assumptions [37]; the present result is a direct comparison with the original fixed-field format, not a claim of the best known redundancy order.

6.2 A strict polynomial saving over single-window coding

A calibrated length-dependent schedule can improve the actual asymptotic rate of a single-window code. This is a comparison of encoded costs, not of two upper bounds. The baseline has literal cost $b > 0$, maximum distance $W = 2^B$, and match cost $B + a(\ell)$, with $a(\ell) \geq 0$. Both formats permit overlapping copies. Put $L_B^{\text{cut}} := \lfloor B/b \rfloor$ and use the schedule

$$w_B(\ell) := \begin{cases} 0, & \ell \leq L_B^{\text{cut}}, \\ B, & \ell > L_B^{\text{cut}}. \end{cases} \quad (45)$$

Thus short matches have distance one and long matches retain the entire history. Supported lengths start at a fixed minimum and are unbounded. Write C_0^* and C_W^* for minimum token costs in the two formats; headers of $o(n)$ bits for fixed B do not affect the rates below.

Proposition 6.4 (Pointwise comparison) *For every input x , schedule (45) satisfies $C_W^*(x) \leq C_0^*(x)$.*

Proof. Replace every baseline match of length $\ell \leq L_B^{\text{cut}}$ by its literals. This cannot increase cost because $B + a(\ell) \geq B \geq b\ell$. All remaining matches retain their full window and original cost in the new format. The decoded history is unchanged. $\square$

The new useful short matches can yield a strictly positive rate gap. The following example uses a single fixed iid source as W grows.

Theorem 6.5 (Strict saving with a polynomial lower bound) *Assume $0 \leq a(\ell) \leq a_0 + a_1 \log_2(\ell + 1)$ for fixed $a_0, a_1 \geq 0$. For any $\beta > 2$, take a binary iid source with $\Pr\{X_i = 1\} = p = 2^{-b\beta}$, with b large enough to encode a binary literal and its discriminator. Under schedule (45), almost surely,*

$$\mathcal{R}_{W,B}^* \leq \mathcal{R}_{0,B}^* - \Omega((\log W)W^{-\beta}), \quad (46)$$

where $\mathcal{R}_{j,B}^* := \limsup_{n \rightarrow \infty} C_j^*(X_1^n)/n$. The implicit positive constant is independent of B .

Proof. Fix $0 < \epsilon < \beta/2 - 1$ and define

$$r_B := \left\lceil \frac{(1 + \epsilon)B}{b\beta} \right\rceil, \quad k_B := L_B^{\text{cut}} - 2r_B.$$

For large B , k_B is a supported positive length. Call a run starting at i good if the preceding W symbols contain no 1^{r_B} , $X_{i-1} = 0$, $X_i^{i+L_B^{\text{cut}}-1} = 1^{L_B^{\text{cut}}}$, and $X_{i+L_B^{\text{cut}}} = 0$. Consider its middle interval $[i + r_B, i + L_B^{\text{cut}} - r_B - 1]$, of length k_B .

No baseline match longer than L_B^{cut} can cover a middle position. Such a match must extend beyond at least one end of the run. If it extends left, copying the first r_B ones of the run would require an occurrence starting at $i - d \geq i - W$. For $d < r_B$, this occurrence crosses the zero at $i - 1$; for $d \geq r_B$, it is a forbidden earlier 1^{r_B} wholly in the preceding window. If the match extends right, it copies the last r_B ones and the following zero. For $d \leq L_B^{\text{cut}}$, that zero has source position $i + L_B^{\text{cut}} - d$ inside the current run, a contradiction. For $d > L_B^{\text{cut}}$, the source of the last r_B ones is wholly in the preceding window, again a forbidden 1^{r_B} . This argument uses source positions at the run boundaries, so it also covers phrases starting before the preceding window and overlapping copies.

Normalize any baseline parse by replacing all matches of length at most L_B^{cut} by literals, as in Proposition 6.4. Every middle interval is now literal. Replace it by a length- k_B , distance-one match, saving

$$s_B := bk_B - a(k_B) = \left(1 - \frac{2(1+\epsilon)}{\beta}\right) B - O(\log B) = \Theta(B) > 0.$$

Good runs, hence the replacement intervals, are disjoint. Therefore, if $N_B(x_1^n)$ counts good runs fully contained in the input with complete preceding windows, this construction proves

$$C_W^*(x_1^n) \leq C_0(P) - s_B N_B(x_1^n) \quad (47)$$

for every baseline parse P , including a minimum-cost one.

Condition on the preceding zero. A union bound on the remaining iid history shows that a good run has probability

$$\pi_B \geq (1-p)^2 p^{L_B^{\text{cut}}} (1 - Wp^{r_B}) \geq (1-p)^2 W^{-\beta} (1 - W^{-\epsilon}).$$

For each fixed B , the ergodic theorem gives $N_B(X_1^n)/n \rightarrow \pi_B$ almost surely. Intersect these probability-one events over integer B , divide (47) by n , and take upper limits. Since $s_B = \Theta(B)$, this proves (46). $\square$

For a finite file, the explicit transformation saves at least $s_B N_B(x_1^n) - \Delta\Gamma$ bits including headers, where $\Delta\Gamma := \Gamma_W - \Gamma_0$ is the header difference. For an independent heuristic parse, the certificate becomes $s_B N_B(x_1^n) - E_W - \Delta\Gamma$, with the excess token cost E_W defined in (6). Each certificate proves strict improvement when it is positive; greediness alone does not control E_W .

For example, $b = 2$, $\beta = 3$, and $p = 1/64$ give a fixed binary source with an actual saving $\Omega((\log W)/W^3)$. This is a polynomial correction to the single-window rate; it does not change the leading positive-entropy redundancy order in $1/\log W$. The short classes in (45) grow with B , unlike a fixed window table. Minimum-bit parsing obtains the theorem, and the explicit transformation of a baseline parse obtains (47) without computing either optimum. The same construction permits any fixed short-distance width w_0 in place of zero, reducing each displayed run saving by only w_0 bits. Thus distance-one windows are a convenient extreme, not essential to the polynomial rate gap. The baseline here uses fixed-width distances; an entropy-coded single-window format is a different comparator.

6.3 Additional gains from single-symbol runs

The ordinary length range can be calibrated to the recurrence scale. At a fixed position of a two-sided finite-alphabet stationary ergodic source with entropy rate $h > 0$, let Λ_W be the longest match available at a backward distance at most W . Inverting the recurrence law $\log_2 R_m/m \rightarrow h$ gives

$$\frac{\Lambda_W}{\log_2 W} \longrightarrow \frac{1}{h} \quad \text{almost surely.} \quad (48)$$

This is the classical match-length consequence of the Ornstein–Weiss theorem [32, 45]. Thus $L_{\text{ord}}(W) \asymp (\log_2 W)/h$ is a natural ordinary cap for a fixed positive-entropy source. The cap in Theorem 6.2 approaches this scale from below, with its coding rate proved explicitly. The constant depends on the source; this argument does not prescribe a single optimal cap for all data. With $W = 2^B$, a fixed-width representation of a length range of order B/h needs $\log_2 B + O(1)$ bits for fixed h , whereas a full-window distance field needs B bits.

For uniform iid data on $q \geq 2$ symbols, $h = \log_2 q$, there is also the elementary fixed-position bound

$$\Pr\{\Lambda_W \geq m\} \leq \min\{1, Wq^{-m}\}, \quad m \geq 1. \quad (49)$$

At a fixed distance d , the match constraints leave d free symbols among $m + d$ iid symbols, so their probability is q^{-m} , even with overlap. A union bound over the W distances proves the result. Consequently matches reaching $\lceil (1 + \epsilon)B/h \rceil$ have probability at most $W^{-\epsilon}$ at a fixed position, for any $\epsilon > 0$. This is a fixed-position statement, not a bound on the longest match in a file. For zero-entropy data, (48) does not apply.

The run-enabled format uses consecutive length symbols $1, 2, 3, \dots, L_{\text{ord}}$. Symbol 1 selects a single-symbol run: $(r, 1)$ is encoded as $(1, r)$, with distance one implicit and r additional copies of the last symbol emitted. The second field is a variable-length count with extensions; its value may exceed L_{ord} ; the selector is not an expanded phrase length of one. Symbols $\ell \geq 2$ give ordinary match lengths, followed by distances decoded with window $W(\ell)$. All other repetitions use those ordinary matches. A fresh run of length $u \geq 2$ needs a seed literal and count $u - 1$; a singleton needs only its literal. Let $g(r)$ include the entire selector and count cost, with unambiguous decoding assumed.

A concrete prefix-token realization uses the Elias gamma code γ :

$$\begin{aligned} \text{binary literal } a : & \quad 0a, \\ \text{ordinary match } (\ell, d) : & \quad 1 \mid \gamma(\ell) \mid \text{bin}_B(d - 1), \quad 2 \leq \ell \leq L_{\text{ord}}(B), \\ \text{run of } r \text{ additional copies} : & \quad 1 \mid \gamma(1) \mid \gamma(r) = 11 \mid \gamma(r). \end{aligned} \quad (50)$$

Since $\gamma(1) = 1$ and $\gamma(\ell)$ starts with zero for $\ell \geq 2$, the ordinary comparator leaves the entire 11 branch unused. Augmentation preserves all its codewords and costs. Replacing a supported distance-one match, $2 \leq r \leq L_{\text{ord}}(B)$, by the run token preserves its expansion and saves exactly

$$[B + 1 + |\gamma(r)|] - [2 + |\gamma(r)|] = B - 1 \quad (51)$$

bits, strictly positive for $B > 1$. Counts beyond the ordinary cap provide the separate benefit of longer expansions. Literal cost is $b = 2$, $a(\ell) = 1 + |\gamma(\ell)|$, and $g(r) = 2 + |\gamma(r)| \leq 3 + 2 \log_2(r + 1)$, uniformly in B . The identity uses this fixed-width-distance construction; an arbitrary Huffman tree need not have unused prefix space, and refitting its codewords

requires charging their changed costs. A particular extension-byte implementation must be evaluated using its actual g .

Proposition 6.6 (Coding by the number of runs) *Let x_1^n have k maximal constant-symbol runs, of lengths $u_1, \dots, u_k$. With literal cost at most λ and framing cost Γ_n , there is a run parse of cost*

$$C_{\text{run}}(x_1^n) \leq \Gamma_n + \lambda k + \sum_{i: u_i \geq 2} g(u_i - 1). \quad (52)$$

If, for all $r \geq 1$, $g(r) \leq A + D \log_2(r + 1)$ with constants $A, D \geq 0$, then

$$C_{\text{run}}(x_1^n) \leq \Gamma_n + (\lambda + A)k + Dk \log_2(n/k). \quad (53)$$

Both bounds also hold for the minimum-cost encoder admitting this parse.

Proof. Encode each run by its seed and remaining count. Since $\sum_i u_i = n$, concavity of the logarithm gives $\sum_i \log_2 u_i \leq k \log_2(n/k)$. Singleton runs need no count. $\square$

The proposition gives a direct additional gain over the original 1977 fixed-field format. Keep the phrase cap $L = L_B^{\text{cap}} = \Theta(B)$ from Theorem 6.2, with $B = \log_2 W$. Each original phrase costs $F_B = B + h \lceil \log_q L \rceil + h = \Theta(B)$ bits and emits at most L source symbols, including its mandatory final symbol. After at most $O(W)$ initialization symbols, the number of phrases is at least $(n - O(W))/L - O(1)$. Hence, when $W \rightarrow \infty$ and $W = o(n)$, $C_{77,B} \geq F_B((n - O(W))/L - O(1)) = \Omega(n)$, with all initialization and framing costs included. This bound holds on every input, without an iid assumption. With logarithmic count costs, $\Gamma_n = O(\log n)$, and $k = O(n^\alpha)$ for fixed $0 \leq \alpha < 1$, it follows that

$$C_{77,B}(x_1^n) = \Omega(n), \quad C_{\text{run}}(x_1^n) = O(n^\alpha \log n) = o(n). \quad (54)$$

For example, $W = 2^{\lceil \rho \log_2 n / h \rceil}$ with fixed $0 < \rho < 1$ gives an admissible joint scaling. The original cap includes its final symbol, so the comparison uses phrase expansion consistently.

This run representation can augment a complete ordinary code while preserving its existing guarantees. For complete prefix codes C_0 and C_{run} on the same input domain, prefix their codewords by a one-bit file selector and choose the shorter branch. The resulting complete code satisfies

$$C_{\text{aug}}(x_1^n) := 1 + \min\{C_0(x_1^n), C_{\text{run}}(x_1^n)\}. \quad (55)$$

All decoder parameters are shared or charged in the branch costs. With $C_0 = C_{77,B}$, this retains the original code within one bit and attains (54) on inputs with few runs. With the capped WLZ code as C_0 , it also preserves the uniform-iid upper bound of Theorem 6.2 while adding this few-run guarantee. Unlike the periodic Huffman comparison, the run branch changes the available phrase expansions. Its gain depends on the specified ordinary cap: the original format can also encode a constant input in $O(\log n)$ bits if allowed an input-sized phrase cap and a fixed window.

More generally, suppose every output-producing token of a baseline format expands to at most L_n symbols and costs at least a fixed $c > 0$ bits. Every baseline parse then satisfies

$$C_{\text{cap}}(x_1^n) \geq c \lceil n/L_n \rceil. \quad (56)$$

Under logarithmic count costs, if $\Gamma_n = O(\log n)$, $k = O(n^\alpha)$, and $L_n = O(n^\delta)$, with $\alpha, \delta \geq 0$ and $\alpha + \delta < 1$, then

$$\frac{C_{\text{run}}(x_1^n)}{n} = O(n^{-(1-\alpha)} \log n), \quad \frac{C_{\text{cap}}(x_1^n)}{n} = \Omega(n^{-\delta}). \quad (57)$$

The run code has a strictly larger decay exponent, up to the logarithm. For logarithmic caps $L_n = O(\log W_n)$ with $2 \leq W_n \leq n^\eta$ and fixed $\eta > 0$, (56) gives $\Omega(n/\log n)$ bits, exceeding the run code's $O(n^\alpha \log n)$ for every fixed $\alpha < 1$. On a constant input with fixed cap L , the corresponding rates are $O(\log n/n)$ and at least c/L . Unrestricted overlapping LZ77 with a logarithmic length code also uses $O(\log n)$ bits on that input: a fixed history alone does not cap match expansion.

The extension rule determines which conclusion applies. Positional variants or universal integer codes can satisfy the logarithmic hypothesis. In contrast, one extension byte per additional 255 symbols, as in the LZ4 match-length format [10], costs $\Theta(r)$ bits. More generally, if $g(r)/r \rightarrow \gamma$ and $\Gamma_n = o(n)$, the single-run parse has limiting rate γ on a constant input. A strict constant improvement still follows whenever $\gamma < c/L$. For one byte per K additional symbols with bounded other overhead, $\gamma = 8/K$. Any fixed run-count cap or compulsory fixed-size block splitting must be charged in (52); it does not satisfy the unbounded-count conclusion (57).

The run token also gives a direct variant of the positive-entropy comparison above, even for some linearly growing count costs. Both formats below have the same ordinary cap, which may be infinite; the comparator reserves binary code space for runs before augmentation.

Corollary 6.7 (Run-token saving on a fixed iid source) *Use the baseline window, token costs, and fixed iid source of Theorem 6.5, with a common ordinary length cap $L_{\text{ord}}(B) \in \{2, 3, \dots\} \cup \{\infty\}$ in both formats. Assume a decodable augmentation retains every supported baseline token at its original cost and admits run tokens at every count $r \geq r_0$, with $g(r) \leq \kappa r + O(\log(r+1))$ for a fixed $0 \leq \kappa < b$. The threshold r_0 and cost constants are independent of B . For minimum token costs, with $o(n)$ framing overhead for fixed B , almost surely,*

$$\mathcal{R}_{\text{run}, B}^* \leq \mathcal{R}_{0, B}^* - \Omega((\log W)W^{-\beta}). \quad (58)$$

Proof. Use the same good runs and middle lengths $k_B = \Theta(B)$ as in Theorem 6.5. Exclusion of long baseline copies uses only the distance bound and remains valid under any ordinary cap. After normalizing the baseline parse, each middle interval is literal and its preceding symbol is one. Replace it by a run token, whose count k_B need not fit below the ordinary cap, saving at least $(b - \kappa)k_B - O(\log B) = \Theta(B)$. The good-run frequency and disjointness argument are unchanged. $\square$

Here $\mathcal{R}_{0, B}^*$ uses the baseline with the chosen common cap; the gap constants are independent of that cap. The prefix construction in (50) satisfies the augmentation hypothesis with $\kappa = 0$ while preserving every ordinary codeword.

Equation (58) improves a polynomial correction; it does not establish a better leading positive-entropy redundancy exponent. Likewise, the few-run condition in (57) describes finite-input classes. For a stationary ergodic source, $k/n \rightarrow \Pr\{X_0 \neq X_1\}$; sublinear run counts therefore restrict such a source to a constant sequence.

Dedicated run tokens have earlier precedents. Pucrunch combines capped LZ77 matches with longer RLE tokens [47], and Linux LZO-RLE reinterprets a reserved distance pattern as a zero-run instruction, targeting data with many zeros such as zram workloads [48]. Pucrunch also restricts two-byte matches to shorter distances than longer matches. These are relevant precedents for special-run coding and length-dependent admissibility. The results here quantify the stated reserved-length variant under explicit cost and parsing assumptions.

WZIP [3] implements this swap in WZIP_L, the variant it uses for inputs of 32 KB or more [4]. A reserved length symbol announces a run of copies of the preceding byte, and the run’s count, not bound by the match cap, is coded in the offset field with that field’s Huffman code and raw extra bits. Runs leave the offset cache unchanged, and consecutive runs merge into one token.

7 Finite-Data Window Selection and Entropy Estimation

7.1 What is being optimized?

For a specified input string x_1^n , let $C_n(\mathbf{W}; x)$ be the complete encoded length under a specified parser and coding format, where $\mathbf{W}$ lists the windows of the length classes. The compression objective under a memory budget M is

$$\min_{\mathbf{W}: \mathbf{M}(\mathbf{W}) \leq M} C_n(\mathbf{W}; x), \quad \mathcal{R}_n := \frac{C_n}{n}, \quad \text{CR}_n := \frac{b_{\text{raw}}}{\mathcal{R}_n}. \quad (59)$$

Here $\mathcal{R}_n$ is bits per input symbol, b_{raw} is the raw storage cost per symbol (8 for bytes), and CR_n is raw size divided by compressed size. Memory includes the indexes of Section 4; a time constraint can be imposed separately. Headers, schedules, dictionaries not already shared with the decoder, and entropy-model costs belong in C_n . The minimizing schedule generally depends on the input and the coding model, not on the data length n alone.

Practical defaults, such as zlib’s 32 KiB history, the largest DEFLATE permits, and Zstandard’s level- and size-dependent parameters [7, 6, 49], are not chosen to minimize this objective for each input. Adaptive selection of LZ77 history lengths also has theoretical precedents for nonstationary sources [50].

If enlarging a window only adds admissible matches while preserving every existing token cost, minimum-bit cost cannot increase: the old parse remains feasible. An interior optimum for compression therefore requires some additional effect, such as window-dependent pointer widths, model overhead, or an imposed resource constraint. A heuristic search budget can also change the actual parse. In particular, the fixed-width cost $\log W$ and the actual variable-cost distances of a modern format are different optimization models.

Selecting among K decoder-known candidate codes costs little: sending the index of the shortest in $\lceil \log K \rceil$ bits, followed by its codeword, is within $\lceil \log K \rceil$ bits of the best candidate on every input. This is standard two-part coding. For the scalar grid $W \in \{1, 2, 4, \dots, 2^{\lfloor \log n \rfloor}\}$, selection costs $O(\log \log n)$ bits. For J nondecreasing integer widths between 0 and B , there are $\binom{B+J}{J}$ schedules, so a shared enumeration costs at most $\lceil \log \binom{B+J}{J} \rceil$

selector bits. The guarantee requires evaluating the candidates on the whole input; selection from a sample needs a separate argument.

7.2 Entropy estimation as an application

The target is the entropy rate $h := \lim_{n \rightarrow \infty} H(P_n)/n$ of an unknown finite-alphabet stationary ergodic source, observed through one sequence X_1^n . Compression-based entropy estimation is well established [51, 52]. Here the contribution is a WLZ-specific consequence: window selection converts the iterated universality limit of Theorem 5.13 into a consistent estimator along the observed sequence, with explicit selection and finite-data costs.

Fix one candidate family, with parsing convention $\text{enc} \in \{\text{greedy}, *\}$, denoting exact greedy or ideal minimum-bit parsing, respectively; Appendix D bounds the time to compute the latter. For each $B \geq 1$, let $C_{B,n}^{\text{enc}}(x_1^n)$ be its complete binary prefix-code length, with maximum history 2^B and the hypotheses of Theorem 5.13. The decoder knows n and the candidate family; initialization and all other required metadata are included in $C_{B,n}^{\text{enc}}$. The parsing convention is fixed in advance, so it requires no selector bit. These are actual code lengths, rather than unconstrained token-cost surrogates. Let b_{raw} be a valid fixed-width raw representation cost per symbol. For $n \geq 2$, choose deterministic integers

$$1 \leq K_n \leq \lfloor \log n \rfloor, \quad K_n \longrightarrow \infty, \quad d_n := \lceil \log K_n \rceil.$$

The simplest choice is $K_n = \lfloor \log n \rfloor$. Define

$$\begin{aligned} \mathcal{L}_n(x_1^n) &:= 1 + \min \left\{ nb_{\text{raw}}, d_n + \min_{1 \leq B \leq K_n} C_{B,n}^{\text{enc}}(x_1^n) \right\}, \\ \hat{h}_n &:= \mathcal{L}_n(X_1^n)/n. \end{aligned} \tag{60}$$

Send a one-bit raw/compressed flag. In the compressed branch, send a minimizing B in d_n bits, followed by its WLZ codeword; fix a deterministic rule for ties. Thus the selected window is $\widehat{W}_n := 2^{\widehat{B}_n}$ in that branch. The raw branch guarantees $\mathcal{L}_n \leq nb_{\text{raw}} + 1$. If n is not shared, its self-delimiting description adds $O(\log(n+1))$ bits to all bounds below.

Theorem 7.1 (Consistency under window selection) *Assume the hypotheses of Theorem 5.13: all lengths from $t_B = o(B)$ through $W = 2^B$ use the full window, fixed match costs are at most $B + a_0 + a_1 \log(\ell+1)$ uniformly in B , and complete initialization and header costs are $o(n)$ for every fixed B . A cap at W is permitted. Exact greedy candidates retain literal fallback and the conceptual-boundary convention. Then, for every finite-alphabet stationary ergodic source,*

$$\hat{h}_n \longrightarrow h \quad \text{almost surely}, \quad \mathbb{E}|\hat{h}_n - h| \longrightarrow 0. \tag{61}$$

Selection costs at most $1 + d_n = O(\log \log n)$ bits beyond the best available WLZ candidate.

Proof. The flag and fixed-width index make $\mathcal{L}_n$ a prefix code for each known n . Every fixed B is eventually among the candidates, so

$$\limsup_{n \rightarrow \infty} \hat{h}_n \leq \inf_{B \geq 1} \limsup_{n \rightarrow \infty} \frac{C_{B,n}^{\text{enc}}(X_1^n)}{n} \leq h \quad \text{almost surely},$$

by Theorem 5.13 and $d_n = o(n)$. For the reverse inequality put $I_n := -\log P_n(X_1^n)$. Kraft's inequality gives, for every $s > 0$,

$$\Pr\{\mathcal{L}_n(X_1^n) < I_n - s\} \leq 2^{-s} \sum_{x_1^n} 2^{-\mathcal{L}_n(x_1^n)} \leq 2^{-s}. \quad (62)$$

Taking $s = 2 \log n$, Borel–Cantelli and the Shannon–McMillan–Breiman theorem give $\liminf_n \hat{h}_n \geq h$ almost surely. Finally $0 \leq \hat{h}_n \leq b_{\text{raw}} + 1/n$; dominated convergence proves convergence in mean absolute error. $\square$

Corollary 7.2 (Finite-data redundancy and estimation error) *The expected redundancies $r_{B,n} := \mathbb{E}C_{B,n}^{\text{enc}}/n - h$ and $\rho_n := \mathbb{E}\hat{h}_n - h$ satisfy*

$$0 \leq \rho_n \leq \frac{1}{n} + \min \left\{ b_{\text{raw}} - h, \frac{d_n}{n} + \min_{1 \leq B \leq K_n} r_{B,n} \right\}. \quad (63)$$

Writing $u_+ := \max\{u, 0\}$, the absolute estimation error satisfies

$$\mathbb{E}|\hat{h}_n - h| \leq \rho_n + 2\mathbb{E}(h - I_n/n)_+ + \frac{2}{n \ln 2}. \quad (64)$$

In particular, for an iid source with single-symbol law p and $\sigma^2 := \text{Var}[-\log p(X_1)]$,

$$\mathbb{E}|\hat{h}_n - h| \leq \rho_n + \frac{\sigma}{\sqrt{n}} + \frac{2}{n \ln 2}. \quad (65)$$

For uniform iid data, $\sigma = 0$.

Proof. The prefix-code entropy bound gives $\mathbb{E}\mathcal{L}_n \geq H(P_n) \geq nh$ and likewise $r_{B,n} \geq 0$. Taking expectations in (60) and using $\mathbb{E} \min_j Z_j \leq \min_j \mathbb{E} Z_j$ proves (63). Integrating (62) gives $\mathbb{E}(I_n - \mathcal{L}_n)_+ \leq 1/\ln 2$. Now use

$$\mathbb{E}|\hat{h}_n - h| = \rho_n + 2\mathbb{E}(h - \hat{h}_n)_+, \quad (h - \hat{h}_n)_+ \leq (h - I_n/n)_+ + (I_n - \mathcal{L}_n)_+/n.$$

For iid data, $\mathbb{E}(I_n/n) = h$ and $\text{Var}(I_n/n) = \sigma^2/n$, so $2\mathbb{E}(h - I_n/n)_+ = \mathbb{E}|I_n/n - h| \leq \sigma/\sqrt{n}$. $\square$

These bounds separate coding redundancy from source-information fluctuations. Any proved finite- n bound for the included complete codes transfers through (63): the selected estimator inherits the smallest such bound over $B \leq K_n$, plus $(1 + d_n)/n$. Other specified prefix codes, such as the cheap-cap family, may be added to the selector. Retaining the universal WLZ subfamily preserves consistency; the index must cover the enlarged list, and a redundancy bound applies only to the format for which it was proved.

The maximum permitted history 2^{K_n} may grow arbitrarily slowly while retaining consistency, because every fixed window eventually becomes available; the selected window itself need not diverge. The result specializes standard universal-code arguments to WLZ. A finite-sample accuracy claim for a corpus requires control of the redundancy and information-fluctuation terms in (64); corpus size alone supplies neither.

The oracle bound can be made quantitative for a fixed nonuniform iid source without adding another compression method to the selector. The following elementary occurrence and concentration argument supplies finite-input bounds for both candidate families.

Proposition 7.3 (Nonuniform iid sources) *Let p be a fixed nonuniform iid law with full support on a finite alphabet of at least two symbols. Put*

$$h := - \sum_a p_a \log p_a > 0, \quad \Delta_p := \log(p_{\max}/p_{\min}) > 0.$$

For either candidate family, put $r_{B,n}^{\text{enc}} := \mathbb{E}C_{B,n}^{\text{enc}}/n - h$. Let $W = 2^B$, let literals cost at most b , and let matches of length m cost at most $B + a(m)$. The deterministic bound $\Gamma(n, B)$ includes all required header and framing costs beyond the literals explicitly charged below. For every $\epsilon > 0$ and integer $t_B \leq m \leq W \leq n$, assuming that every distance through W is allowed at length m , define

$$e(W, m, \epsilon) := \exp\left(-\frac{2m\epsilon^2}{\Delta_p^2}\right) + \exp\left(-\left\lfloor \frac{W}{m} \right\rfloor 2^{-m(h+\epsilon)}\right).$$

The minimum-bit candidate satisfies

$$0 \leq r_{B,n}^* \leq \frac{B + a(m)}{m} - h + b e(W, m, \epsilon) + \frac{b(W + m) + \Gamma(n, B)}{n}. \quad (66)$$

For exact greedy candidates satisfying Lemma 5.12, with $\bar{K}_m$ from Theorem 5.13,

$$\begin{aligned} 0 \leq r_{B,n}^{\text{greedy}} \leq & \frac{\bar{K}_m + b(t_B - 1)}{m} - h + b e(W, m, \epsilon) \\ & + \frac{2\bar{K}_m + b(W + m) + \Gamma(n, B)}{n}. \end{aligned} \quad (67)$$

For $n < W$, literal fallback instead gives $r_{B,n}^{\text{enc}} \leq b - h + \Gamma(n, B)/n$. If $t_B = o(B)$ and $a(m) = O(\log(m + 1))$ uniformly in B , then, uniformly for sufficiently large B and $n \geq 2^B$,

$$r_{B,n}^* \leq O\left(\sqrt{\frac{\ln B}{B}}\right) + O\left(\frac{2^B + B}{n}\right) + \frac{\Gamma(n, B)}{n}. \quad (68)$$

For exact greedy candidates, the same bound has the additional term $O(t_B/B)$. The constants may depend on the fixed law p and code-cost bounds. For the minimum-bit selector with $K_n = \lfloor \log n \rfloor$, if $\Gamma(n, B_n) = O(\log n)$ at $B_n = \lfloor (\log n)/2 \rfloor$, then

$$\mathbb{E}|\hat{h}_n - h| = O\left(\sqrt{\frac{\log \log n}{\log n}}\right). \quad (69)$$

For the exact greedy selector, the bound in (69) has the additional term $O(t_{B_n}/B_n)$. In particular, the same square-root order holds when $t_B = O(\sqrt{B \ln B})$; the weaker $t_B = o(B)$ still ensures consistency.

Proof. For a comparison parse, send the first W symbols as literals, then process deterministic consecutive blocks of length m ; send the final fewer than m symbols as literals. At each full block, use one match if its word occurs among $N := \lfloor W/m \rfloor$ disjoint length- m blocks in

the preceding window, and otherwise use literals. All these matches are admissible. The fixed block starts, unlike adaptively selected greedy starts, permit direct use of the iid law. No independence between the success events at different block starts is needed. The chosen history blocks end before the query begins and have starting distances at least m and at most W . Thus this comparison uses only nonoverlapping copies, although the format permits overlap.

The query word is independent of its preceding window. Conditional on query u , the selected past words are independent, each with probability $P_m(u)$ of equaling u . Their failure probability is at most $(1 - P_m(u))^N \leq \exp(-NP_m(u))$. The query self-information $I_m := -\log P_m(X_1^m)$ is a sum of independent variables with mean h and range length Δ_p . Hoeffding's inequality [53, Theorem 2] gives

$$\Pr\{I_m > m(h + \epsilon)\} \leq \exp(-2m\epsilon^2/\Delta_p^2).$$

On the complementary event, $P_m(X_1^m) \geq 2^{-m(h+\epsilon)}$. Splitting the failure probability over these events proves the bound $e(W, m, \epsilon)$. An upper bound exceeding one is harmless.

Write $n - W = Jm + r$, where $0 \leq r < m$. Each full block has expected cost at most $B + a(m) + bm e(W, m, \epsilon)$. Thus the complete comparison code costs at most

$$bW + J\{B + a(m) + bm e(W, m, \epsilon)\} + br + \Gamma(n, B)$$

in expectation. Minimum-bit WLZ costs no more. Using $Jm \leq n$ and $r < m$ proves (66); the lower bound follows from the prefix-code entropy inequality. In particular, the startup charge belongs to this comparison parse and is not asserted to be intrinsic to every WLZ encoding.

For exact greedy parsing, split the first W symbols into full m -blocks and at most one shorter block, and designate them bad. Partition the remaining symbols into full m -blocks and a possibly shorter bad terminal block. There are at most $n/m + 2$ reference blocks, at most n/m good blocks, and an expected bad-symbol count of at most $W + m + ne(W, m, \epsilon)$. Apply Lemma 5.12; since every reference block has length at most m , its logarithmic term is at most $\bar{K}_m$ per block. This proves (67), without independence assumptions at greedy starts. Here the initial-window charge belongs to the analysis partition, not to a required raw startup in the encoder.

For the asymptotic choice, take

$$\epsilon_B = \Delta_p \sqrt{\frac{2h \ln B}{B}}, \quad m_B = \left\lfloor \frac{B - 3 \log B}{h + \epsilon_B} \right\rfloor.$$

Since $m_B \sim B/h$, the assumed full-window tail admits this length eventually. Also $hm_B/B \geq 1/2$ for sufficiently large B , so the first failure term is at most B^{-2} . For the second,

$$\left\lfloor \frac{W}{m_B} \right\rfloor 2^{-m_B(h+\epsilon_B)} \geq \frac{B^3}{2m_B} = \Omega(B^2).$$

The floor in m_B and the length-code cost contribute $O((\log B)/B)$ to $(B + a(m_B))/m_B - h$, leaving $O(\sqrt{(\ln B)/B})$. Substitution proves (68). In the greedy bound, $\bar{K}_{m_B} = B + O(\log B)$ and $b(t_B - 1)/m_B = O(t_B/B)$; the remaining boundary terms are $O((2^B + B)/n)$.

Finally the default candidate list contains B_n , with 2^{B_n} within a factor two of $\sqrt{n}$. Its redundancy bound, the selector overhead $O((\log \log n)/n)$, and the iid information-fluctuation term $O(n^{-1/2})$ in Corollary 7.2 imply (69), with the stated additional $O(t_{B_n}/B_n)$ term for greedy candidates. $\square$

The source-dependent length m_B specifies only a comparison parse or analysis partition in the proof; neither candidate encoder nor the window selector needs to know p , h , or Δ_p . Every match already transmits its own length in the WLZ token code; the comparison therefore requires no separate block-size message or source-probability model. The literals and matches use the original candidate’s decoder throughout. The rate requires the stated candidate list and a header bound along $B = B_n$; fixed-window $o(n)$ headers or an arbitrarily slowly growing candidate list alone do not supply it. The guarantee is for each fixed nonuniform iid law. The uniform case $\Delta_p = 0$ is covered separately by the preceding uniform-source analysis.

8 Conclusion

WLZ can restrict short-match histories while retaining established convergence orders and exact-greedy stationary-ergodic universality under the stated assumptions. The transfer framework quantifies parsing losses and distinguishes gains from the window schedule, length coding, extended single-symbol runs, and entropy coding of the fields. With fixed-width distance fields the windows save bits directly, but their asymptotic strict gain on the sources analyzed is polynomially small; the larger gains come from the token code. In the reported WZIP configuration, ablations indicate that length-conditioned offset coding improves compression, while restricted windows reduce indexing resources [3].

A principal theoretical direction is to obtain matching upper and lower bounds for the fixed-source saving from calibrated windows, keeping the length-code model fixed. This would determine whether the proved polynomial saving is sharp and help identify conditions for improving the leading redundancy term. Extending the quantitative analysis to dependent sources also calls for quantitative recurrence-tail and search-loss bounds.

An algorithmic goal is efficient joint selection of window sizes and hash-chain inspection budgets under memory and time constraints, related to bicriteria parsing, which trades compressed size against decompression time [54]. The target is an approximation guarantee for complete encoded length, including model and selector costs, without exhaustively encoding every candidate. Combining such a guarantee with quantitative source bounds would help extend the entropy-estimation analysis to practical encoders.

A A Positive-Entropy Redundancy Obstruction

This appendix concerns a specified analytical code. It explains why changing the short-match windows alone cannot establish the desired exponent in that model.

Theorem A.1 (Fixed-threshold, separately coded lengths) *Let the input be iid uniform on $q \geq 2$ symbols, with entropy $h = \log q$. Let $W = 2^B$. Suppose the exact greedy parser has a fixed integer threshold $t \geq 1$ independent of B , assigns every length $\ell \geq t$ the full window and exactly B distance bits, and supports unbounded overlapping matches. Shorter supported lengths may use any smaller windows. Suppose a long match costs $B + a(\ell)$, where*

$$a(\ell) \geq c \log(\ell + 1) - C, \quad c > 0,$$

and all other costs are nonnegative. The initial W symbols are emitted before greedy parsing. Phrase boundaries are determined by exact longest admissible matching, without future-dependent parse optimization. Then

$$\liminf_{n \rightarrow \infty} \frac{\mathbb{E} C_B(X_1^n)}{n} \geq h + ch \frac{\log B}{B} - O(B^{-1}). \quad (70)$$

The same conclusion permits replacing a conceptual matched phrase by its literals at fixed cost $b\ell$ if $b > h$, provided its conceptual greedy boundary is retained.

Proof. First bound the mean phrase length while accounting for lookahead. Extend the file to an infinite iid sequence. Let S_j and L_j be the successive conceptual phrase starts and lengths of the infinite parse, and let $\mathcal{F}_i$ reveal all symbols through position i . The time $T_j := S_j + t - 1$ is a stopping time. Indeed, if the full-window maximum is at least t , it is also the longest admissible match, and its decision requires only its first mismatch symbol. If that maximum is shorter than t , at most a fixed t -symbol prefix is needed to resolve all shorter classes. A long phrase ending immediately before i is resolved by symbol i ; a shorter phrase beginning at $s < i$ is resolved by symbols through $s + t - 1 \leq i + t - 1$. Induction makes $\{S_j = i\}$ measurable in $\mathcal{F}_{i+t-1}$, proving the stopping-time assertion.

Condition on $\mathcal{F}_{T_j}$. For any fixed distance $d \leq W$, a match of length $r > t$ requires $r - t$ fresh iid symbols to equal previously determined symbols. Overlap does not change the probability: when each new symbol is exposed, its required predecessor has already been determined. A union bound over distances yields

$$\Pr\{L_j \geq r \mid \mathcal{F}_{T_j}\} \leq \min\{1, Wq^{-(r-t)}\}, \quad r > t.$$

Summing these tails gives

$$\mathbb{E}[L_j \mid \mathcal{F}_{T_j}] \leq \mu_B := t + \lceil B/h \rceil + (q - 1)^{-1} = B/h + O(1). \quad (71)$$

Let N count infinite-parse starts in $[W + 1, n]$. Truncating their lengths at the file boundary makes the sum exactly $n - W$. Multiplying the conditional bound by the indicator that the corresponding start lies in the file and summing at most $n - W$ terms gives

$$\mathbb{E}N \geq (n - W)/\mu_B. \quad (72)$$

This step does not assume independent phrase lengths.

Choose a constant $0 < \theta < 1$, and put $m = \lfloor \theta B/h \rfloor$. A past window contains $\lfloor W/m \rfloor$ independent disjoint length- m blocks. The probability that it fails to contain every one of the q^m possible words is at most

$$\delta_B := q^m \exp\{-\lfloor W/m \rfloor q^{-m}\}. \quad (73)$$

This decreases faster than every inverse power of B . At a position where full coverage holds and at least m symbols remain, the greedy phrase has length at least m . For large B this is in the full-window class and costs at least

$$J_B := B + c \log(m + 1) - C = B + c \log B - O(1).$$

The finite parser agrees with these infinite-parse starts until its terminal region: a match truncated to length at least t remains legal, and any discrepancy below t occurs in the final $t - 1$ symbols. Consequently every covered infinite-parse start with at least $m \geq t$ symbols remaining yields a finite-file phrase with the stated cost. The number of phrase starts without coverage is bounded by the number of such source positions, whose expectation is at most $(n - W)\delta_B$. At most m additional starts lie in the terminal region. Therefore

$$\mathbb{E}C_B \geq J_B [(n - W)/\mu_B - (n - W)\delta_B - m].$$

Divide by n , let $n \rightarrow \infty$, and expand J_B/μ_B to obtain (70).

For the permitted literal fallback, choose $h/b < \theta < 1$. Then $bm > J_B$ for large B , so both possible representations of a covered long phrase have cost at least J_B . The same proof applies. $\square$

The theorem rules out redundancy $O(B^{-p})$ for $p > 1$, and even $O(B^{-1})$, in this model. Its length-cost assumption holds for common universal integer codes, but does not follow from prefix-decodability alone. An adaptive length model can give small code lengths near the typical match length without satisfying that pointwise lower bound. A raw block coded at h bits per symbol on this uniform source also defeats the obstruction, as it should. Arbitrary minimum-bit parsing, unbounded lookahead decisions, and reparsing after a rejected long match are not covered. Thus a stronger exponent for practical WLZ would require a proof about its long-phrase entropy model and parsing policy, not merely its short-match windows.

B Implementation Considerations

Length-specific indexes permit parallel match search. At a fixed input position, each index can traverse its own chain concurrently against a consistent history, using its class-specific stopping rule (Section 3). If search g , including verification and extension, takes time T_g , one worker per index gives an ideal search time of $\max_g T_g$ plus candidate selection, in place of $\sum_g T_g$ for serial execution. Selection follows the stated length and distance rules rather than completion order, so the encoded result is unchanged. Index updates, chain traversal, workload imbalance, and shared memory bandwidth limit the actual gain, and the chosen phrase still determines the next parsing position. Parallel length-specific dictionaries have precedents in restricted parsing [55]; an optimized 842 implementation also uses separate length-specific hash tables and vectorized hashing [56].

Seed lengths, retained histories, and bucket counts can also balance the expected bucket occupancies of these searches. In the notation of Section 4.4, let group g retain V_g seeds of length k_g in K_g buckets, and let Q_g be the number of prior nodes in its query bucket at a fixed position. For uniform iid data on q symbols and input-independent hashes with

unequal-seed collision probability at most $1/K_g$,

$$V_g q^{-k_g} \leq \mathbb{E}_{X, \text{hash}} Q_g \leq V_g q^{-k_g} + \frac{V_g}{K_g},$$

since each prior seed equals the query with probability q^{-k_g} , including overlaps. Choosing $V_g q^{-k_g} = \Theta(\lambda)$ and $V_g/K_g = O(\lambda)$ for a common $\lambda > 0$ gives comparable expected occupancies: short seeds use short histories, while long seeds admit larger ones. Comparable occupancies do not guarantee equal search times.

Preset dictionaries, as supported by zlib, Zstandard, and LZ4 [7, 49, 57], fit this organization. Number input symbols from zero and a shared dictionary of length L_D from $-L_D$ through -1 . A match at input position i then has distance $1 \leq d \leq \min\{W(\ell), i + L_D\}$, so the dictionary supplies initial history under the same length-dependent limits, and prebuilt length-specific indexes can be reused across inputs. The decoder needs the dictionary but not the encoder's indexes; dictionary identification and delivery costs must be charged.

C Proof of the Significant-Bit Coding Bound

We specialize the uniform-prefix mixture argument of Ryabko, Astola, and Egiastian [42, Thm. 1 and Appendix]. For a fixed partition, $\Delta = \mathbb{E}r_J - H(X \mid J)$ is convex in the distribution, since conditional entropy is concave [43, Thm. 2.1]; it is nonnegative since a bucket has at most 2^{r_J} valid values. Separate the exact head and cap atoms. Set $a_m := p_m$ for $k \leq m < c$ and $a_c := 0$. The tail is a mixture of uniforms $U[k, m]$, with

$$w_m := (m - k + 1)(a_m - a_{m+1}) \geq 0, \quad \sum_{m=k}^{c-1} w_m = \tau.$$

Each exact atom has zero loss. For $U[k, m]$, the loss is zero if $m < T$. Otherwise all occupied buckets except the last are complete. If that last bucket has capacity s and t occupied values, then $s = 2^{\lfloor \log m \rfloor - \nu + 1} \leq 2m/T$ and

$$\Delta(U[k, m]) = \frac{t}{m - k + 1} \log \frac{s}{t} \leq \frac{s}{e \ln 2 (m - k + 1)} \leq \frac{2}{e \ln 2 (T - k + 1)}.$$

Here $\max_{0 < z \leq 1} z \log(1/z) = 1/(e \ln 2)$ and $m/(m - k + 1) \leq T/(T - k + 1)$ for $m \geq T$. Convexity gives (13), including a partially used last bucket immediately before the exact cap. Huffman coding of labels costs less than $H(J) + 1$ bits per value; whole-value Huffman coding costs at least $H(X)$ and is no worse than the grouped prefix code. Use zero-bit singleton codes in both cases. Summing over the three empirical streams gives (14).

For the stated Zstandard comparison, a uniform positive prefix $U[1, m]$ ending in a dyadic bucket $[s, 2s - 1]$ has loss $t \log(s/t)/m \leq 1/(e \ln 2)$, since $s \leq m$. Mixtures give the same bound for decreasing positive probabilities; an arbitrary atom at zero contributes no loss. Refining a bucket cannot increase the grouping loss, by nonnegativity of relative entropy between its refined-label probabilities and relative bucket sizes. The cited Zstandard tables are such refinements, including the ordinary dyadic coded-offset table. This argument concerns the emitted numeric histograms, not an assumed source distribution. $\square$

D Exact Minimum-Cost Parsing

The main results concern exact greedy parsing and the minimum token cost C_W^* as $n \rightarrow \infty$. This appendix concerns $C_W^*(x_1^n)$ for a given input: which single-window shortcuts fail under length-dependent windows, and how quickly C_W^* and a minimizing parse can be computed. A parse is a sequence of phrases, each a literal of cost b or an admissible match of length at most a cap L ; phrase counts include literals. A phrase of length ℓ starting at position i ends before $i + \ell$.

D.1 Two single-window properties that fail

Under one window, a suffix of an admissible match is an admissible match at the same distance. If every length from one is supported, exact greedy parsing then minimizes the number of phrases. Let greedy and another parse have their t th phrases at $g_t \geq p_t$. Either $g_t \geq p_{t+1}$, or the other parse's t th phrase covers g_t and its suffix from g_t is a phrase ending at p_{t+1} ; in both cases $g_{t+1} \geq p_{t+1}$ (see [24] for the general closure conditions). Likewise, for match costs nondecreasing in length, some minimum-cost parse uses only maximal matches, the longest of each cost from each position [25]. Both arguments shorten a match from the left. Under length-dependent windows the shortened match may fall below t_B and lie outside its own window, the loss of suffix closure that Proposition 5.3 charges, and both properties fail.

Example 1 (phrase count). Let lengths at most 2 have window 4 and longer lengths the whole history. After the history `abcx#xyz#`, exact greedy parsing codes `abcxyz` as `abcx`, `y`, `z`: the longest admissible match `abcx` leaves `yz`, whose only earlier copy lies 7 symbols back. The parse `abc, xyz` has two phrases.

Example 2 (maximal matches). Let $x_1^{12} = 010000001000$, $L = 8$, window 1 for lengths 2–4 and window 8 for lengths 5–8, literal cost $b = 2$, and match cost 7. Three literals, the match (4, 1) and the match (5, 7) cost 20 bits, the minimum. Keeping only the longest admissible match at each position, here (5, 1) at position 4, leaves 1000, whose only earlier copy lies at distance 7 and is admissible only for lengths of at least 5; the best parse from literals and longest matches costs 21 bits. With window 8 for every length, both minima are 20 bits. Figure 4 shows these parses.

D.2 Length-only token costs

Under the cost model (1), a match costs $c(\ell) = a(\ell) + w(\ell)$, independent of its distance. The schedule need not be monotone. Partition the supported lengths into the K maximal intervals $[s_k, e_k]$ on which $W(\ell)$ is constant, equal to W_k , with $e_K = L$; for a nondecreasing schedule, K is the number of distinct windows. For a position i , let $\lambda_k(i)$ be the largest $\ell \leq \min\{L, n - i + 1\}$ such that $x_i^{i+\ell-1}$ has a copy at a distance $d \leq \min\{W_k, i - 1\}$, or 0 if there is none.

Lemma D.1 (Admissible lengths) *The admissible match lengths at position i are exactly $\bigcup_{k=1}^K [s_k, \min\{e_k, \lambda_k(i)\}]$.*

	1	2	3	4	5	6	7	8	9	10	11	12
	0	1	0	0	0	0	0	0	1	0	0	0
minimum cost	0	1	0	(4, 1)				(5, 7)				20 bits
longest match only	0	1	0	(5, 1)				1	0	0	0	21 bits
one window, longest match	0	1	0	(5, 1)				(4, 7)				20 bits

Figure 4: Example 2 as parses. With window 1 for lengths 2–4 and window 8 for lengths 5–8 (literals 2 bits, shaded; matches 7 bits), the admissible lengths at position 4 are $[2, 4] \cup [5, 5]$ (Lemma D.1). The minimum-cost parse uses the shorter match (4, 1); taking the longest match (5, 1) leaves 1000, whose only copy, at distance 7, is admissible only for lengths of at least 5. With one window of 8 for every length, the longest-match parse is again optimal.

Proof. If $s_k \leq \ell \leq \min\{e_k, \lambda_k(i)\}$, the first ℓ symbols of the copy defining $\lambda_k(i)$ form a length- ℓ match at a distance at most $W_k = W(\ell)$. Conversely, an admissible length- ℓ match with $\ell \in [s_k, e_k]$ has distance at most W_k , so $\lambda_k(i) \geq \ell$. $\square$

Theorem D.2 (Exact parsing with length-only costs) *Let Y be the number of maximal runs on which $a(\ell)$ is constant, over the supported lengths up to $\min\{L, n\}$. For a finite alphabet, $C_W^*(x_1^n)$ and a minimizing parse can be computed offline in $O(n(K + Y) \log n)$ time using $O(n)$ words of space.*

Proof. Build the suffix array of x_1^n , its inverse, and the longest-common-prefix array with a constant-time range-minimum structure, all in $O(n)$ time [58, 59, 60]; the longest common prefix of any two suffixes is then available in $O(1)$ time. Comparing suffixes also covers overlapping copies. Process $i = 1, \dots, n$ in order. A maximum segment tree over suffix ranks stores each earlier position at its suffix’s rank. For each k , descending this tree finds the nearest ranks below and above that of suffix i whose stored positions are at least $i - W_k$. Among the positions within distance W_k , the suffix sharing the longest prefix with suffix i is at one of these two ranks, so $\lambda_k(i)$, capped at $\min\{L, n - i + 1\}$, takes $O(\log n)$ time. Position i is then inserted.

Let $f(i)$ be the minimum cost of parsing x_1^{i-1} , with $f(1) = 0$; the target is $f(n + 1)$. All phrases ending at $i - 1$ start earlier, so $f(i)$ is final when position i is processed. Relax $f(i + 1)$ by $f(i) + b$. The class boundaries s_k and the run boundaries of a cut the supported lengths into at most $K + Y$ segments of constant cost. By Lemma D.1, the admissible lengths at i meet each segment in an interval $[u, v]$, possibly empty; relax $f(j)$ by $f(i) + c(u)$ for all $j \in [i + u, i + v]$. A segment tree with lazy range minimum updates and point queries performs each relaxation and each read of $f(i)$ in $O(\log n)$ time, and stores with each value the phrase that attains it, for backtracking. $\square$

The algorithm works offline on the whole input with suffix-array structures, separately from the hash indexes of Section 4.4. For $K = 1$ it is the single-window computation; length-dependent windows add at most $K - 1$ segments per position. Under the hypotheses of Theorem 5.13, all lengths of at least t_B share the full window, so $K \leq t_B$. The code-cost bound there limits $a(\ell)$ but not the number Y of its changes. For a length code that is

constant on dyadic length ranges, $Y = O(\log L)$; with $L = W$ the time is $O(n(t_B + B) \log n) = O(nB \log n)$ for $t_B = o(B)$, and the minimum-bit variant of the entropy estimator (60), which evaluates $C_{B,n}^*$ for every $B \leq K_n \leq \log n$, takes $O(n \log^3 n)$ time.

D.3 Distance-dependent costs

When a match cost $c(\ell, d)$ also depends on the distance, as with variable-length distance codes, a constant-cost segment of lengths no longer determines the cost. Example 2 shows that maximal matches do not suffice. For fixed costs that are nondecreasing in match length, a bounded number of shorter matches near each maximal one does. Arbitrary fitted Huffman costs need not be monotone, and neither is the cheap cap token of Theorem 6.2; such costs need separate treatment, although with length-only costs they remain covered by Theorem D.2.

Proposition D.3 (Pruned parsing graph) *Suppose that every length of at least t_B has the full window, that $c(\ell, d) \geq 0$ is nondecreasing in ℓ for each d , and that $b \geq 0$. At each position i , keep*

- (a) *for each $\ell < t_B$, one cheapest admissible match of length ℓ ;*
- (b) *for each cost q of a match starting at i , a longest admissible match (ℓ', d') of cost at most q , and its prefixes at distance d' of every length $\ell \geq t_B$ with $\ell' - t_B < \ell \leq \ell'$.*

Some minimum-cost parse uses only literals and kept matches. With at most Q distinct costs per position, at most $t_B - 1 + Qt_B$ matches are kept per position.

Proof. Prefixes of length at least t_B are admissible, and by monotonicity a prefix in (b) costs at most q . Take a minimum-cost parse whose matches before i are kept, and let e be its match at i , of length ℓ and cost q . If $\ell < t_B$, replace e by the kept match of length ℓ . Otherwise let (ℓ', d') be the match of (b) for q , so $\ell' \geq \ell \geq t_B$; if $\ell' = \ell$, replace e by it. If $\ell' > \ell$, let f be the phrase covering position $i + \ell' - 1$; it starts at some $k \geq i + \ell$ and ends before m . If $m = i + \ell'$ or $m - i - \ell' \geq t_B$, replace the phrases from e through f by (ℓ', d') , followed, if $m > i + \ell'$, by the suffix of f from $i + \ell'$: a match at the distance of f , of length at least t_B and cost at most that of f . If $0 < m - i - \ell' < t_B$ and f has length at least t_B , put $j = m - t_B$, so that $k \leq j < i + \ell'$; replace the phrases from e through f by the prefix of (ℓ', d') ending before j and the length- t_B suffix of f from j . If $0 < m - i - \ell' < t_B$ and f is shorter than t_B , then $k > m - t_B > i + \ell' - t_B$; replace the phrases from e up to f by the prefix of (ℓ', d') ending before k , and keep f . Each prefix used has length between ℓ and ℓ' and more than $\ell' - t_B$, so it is kept. The removed phrases have nonnegative cost, so the parse remains minimal, and its match at i is now kept. Induction on i completes the proof. $\square$

For $t_B = 1$, (a) is empty and (b) keeps exactly the maximal matches of the single-window case [25]. Under length-dependent windows the graph grows by a factor of at most t_B , which is $o(B)$ under the hypotheses of Theorem 5.13. Generating the kept matches on the fly, as [25] does for one window, is not addressed here; the per-cost-class sliding-window suffix trees of [22] are a natural starting point. Neither result covers repeat-offset caches or adaptively updated prices, whose costs depend on the parse history.

References

- [1] Y. Wu, “Deflate compression algorithm,” US Patent 9,577,665 B2, February 21, 2017.
- [2] Y. Wu, “Improved LZ77 compression,” in *2021 Data Compression Conference (DCC)*, Snowbird, UT, USA, 2021, p. 377. doi:10.1109/DCC50243.2021.00066.
- [3] Y. Wu, “WZIP and WLZ4: LZ77 codecs with match-length-dependent sliding windows,” submitted to *2027 Data Compression Conference (DCC)*, 2026. Preprint: https://github.com/icodywu/WZIP/blob/main/papers/WZIP_WLZ4_DCC.pdf.
- [4] Y. Wu, “WZIP and WLZ4 source code,” 2026. <https://github.com/icodywu/WZIP>.
- [5] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977. doi:10.1109/TIT.1977.1055714.
- [6] P. Deutsch, “DEFLATE compressed data format specification version 1.3,” RFC 1951, 1996. <https://www.rfc-editor.org/rfc/rfc1951>.
- [7] J.-l. Gailly and M. Adler, “zlib 1.3.1 manual,” 2024, <https://zlib.net/manual.html>.
- [8] Y. Collet and M. Kucherawy, “Zstandard compression and the ‘application/zstd’ media type,” RFC 8878, February 2021. doi:10.17487/RFC8878.
- [9] Y. Collet and contributors, “Zstandard: Fast real-time compression algorithm,” README.md, release v1.5.7, 2025, benchmark table (lzbenc on the Silesia corpus). <https://github.com/facebook/zstd/blob/v1.5.7/README.md>.
- [10] Y. Collet and contributors, “LZ4 block format description,” revised July 31, 2022, LZ4 project documentation. https://github.com/lz4/lz4/blob/dev/doc/lz4_Block_format.md.
- [11] Y. Collet and contributors, “Zstandard—Fast real-time compression algorithm,” project documentation, section “Status.” <https://github.com/facebook/zstd>. Accessed October 1, 2026.
- [12] Y. Collet and contributors, “LZ4—Extremely fast compression,” project website, section “LZ4 is used by.” <https://lz4.org/>. Accessed October 1, 2026.
- [13] Y. Collet and contributors, “Zstandard compression-level parameters and optimal parsing implementation,” release v1.5.7, 2025, `lib/compress/clevels.h` and `lib/compress/zstd_opt.c`. <https://github.com/facebook/zstd/tree/v1.5.7/lib/compress>.
- [14] Y. Collet and contributors, “Zstandard v1.5.7 release notes,” February 19, 2025. <https://github.com/facebook/zstd/releases/tag/v1.5.7>.
- [15] Google, “Snappy compressed format description,” revised October 5, 2011, `format_description.txt`, Google Snappy repository. <https://github.com/google/snappy>.

- [16] I. Pavlov, “LZMA specification (draft version),” June 14, 2015, section “The match distance decoding.” <https://www.7-zip.org/a/lzma-specification.7z>.
- [17] J. Alakuijala and Z. Szabadka, “Brotli compressed data format,” RFC 7932, July 2016, Secs. 7.2–7.3. doi:10.17487/RFC7932.
- [18] P. Skibiński, “Lizard v1.x block format,” revised October 8, 2016, `doc/lizard_Block_format.md`, Lizard repository. <https://github.com/inikep/lizard>.
- [19] Y. Shavidze, “What governs decode throughput in absolute-offset GPU LZ77? A work-granularity mechanism and an encode-time min-match-length lever,” arXiv:2607.18541v1, July 20, 2026, preprint. <https://arxiv.org/abs/2607.18541v1>.
- [20] K. Sadakane and H. Imai, “Improving the speed of LZ77 compression by hashing and suffix sorting,” *IEICE Transactions on Fundamentals*, vol. E83-A, no. 12, pp. 2689–2698, 2000.
- [21] Y. Collet and contributors, “Zstandard double-fast match finder,” `lib/compress/zstd_double_fast.c`, release v1.4.4, November 5, 2019. <https://github.com/facebook/zstd>.
- [22] M. Crochemore, A. Langiu, and F. Mignosi, “The rightmost equal-cost position problem,” in *2013 Data Compression Conference (DCC)*, pp. 421–430, 2013. doi:10.1109/DCC.2013.50.
- [23] D. T. Hoang, P. M. Long, and J. S. Vitter, “Dictionary selection using partial matching,” *Information Sciences*, vol. 119, nos. 1–2, pp. 57–72, 1999. doi:10.1016/S0020-0255(99)00060-2.
- [24] M. Crochemore, A. Langiu, and F. Mignosi, “Note on the greedy parsing optimality for dictionary-based text compression,” *Theoretical Computer Science*, vol. 525, pp. 55–59, 2014. doi:10.1016/j.tcs.2014.01.013.
- [25] P. Ferragina, I. Nitto, and R. Venturini, “On the bit-complexity of Lempel-Ziv compression,” *SIAM Journal on Computing*, vol. 42, no. 4, pp. 1521–1541, 2013. doi:10.1137/120869511.
- [26] M. Crochemore, L. Giambruno, A. Langiu, F. Mignosi, and A. Restivo, “Dictionary-symbolwise flexible parsing,” *Journal of Discrete Algorithms*, vol. 14, pp. 74–90, 2012. doi:10.1016/j.jda.2011.12.021.
- [27] D. Kosolobov, “Relations between greedy and bit-optimal LZ77 encodings,” in *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018)*, LIPIcs, vol. 96, pp. 46:1–46:14, 2018. doi:10.4230/LIPIcs.STACS.2018.46.
- [28] J. Ellert, J. Fischer, and M. R. Pedersen, “New advances in rightmost Lempel-Ziv,” in *String Processing and Information Retrieval (SPIRE 2023)*, LNCS, vol. 14240, pp. 188–202, 2023. doi:10.1007/978-3-031-43980-3_15.

- [29] J. Ellert, “Sublinear time Lempel–Ziv (LZ77) factorization,” in *String Processing and Information Retrieval (SPIRE 2023)*, Lecture Notes in Computer Science, vol. 14240, pp. 171–187, 2023. doi:10.1007/978-3-031-43980-3_14.
- [30] D. Kempa and T. Kociumaka, “Lempel-Ziv (LZ77) factorization in sublinear time,” in *65th IEEE Symposium on Foundations of Computer Science (FOCS 2024)*, pp. 2045–2055, 2024. doi:10.1109/FOCS61266.2024.00122.
- [31] A. D. Wyner and J. Ziv, “The sliding-window Lempel-Ziv algorithm is asymptotically optimal,” *Proceedings of the IEEE*, vol. 82, no. 6, pp. 872–877, 1994. doi:10.1109/5.286191.
- [32] D. S. Ornstein and B. Weiss, “Entropy and data compression schemes,” *IEEE Transactions on Information Theory*, vol. 39, no. 1, pp. 78–83, 1993. doi:10.1109/18.179344.
- [33] T. Jacob and R. K. Bansal, “Almost sure optimality of sliding window Lempel-Ziv algorithm and variants revisited,” *IEEE Transactions on Information Theory*, vol. 59, no. 8, pp. 4977–4984, 2013. doi:10.1109/TIT.2013.2259291.
- [34] S. Raskhodnikova, D. Ron, R. Rubinfeld, and A. D. Smith, “Sublinear algorithms for approximating string compressibility,” *Algorithmica*, vol. 65, no. 3, pp. 685–709, 2013. doi:10.1007/s00453-012-9618-6.
- [35] T. Kociumaka, G. Navarro, and N. Prezza, “Toward a definitive compressibility measure for repetitive sequences,” *IEEE Transactions on Information Theory*, vol. 69, no. 4, pp. 2074–2092, 2023, Lemma II.4. doi:10.1109/TIT.2022.3224382.
- [36] M. Abadi, V. Amorim, and S. Gallo, “Potential well in Poincaré recurrence,” *Entropy*, vol. 23, no. 3, article 379, 2021. doi:10.3390/e23030379.
- [37] A. D. Wyner and A. J. Wyner, “Improved redundancy of a version of the Lempel-Ziv algorithm,” *IEEE Transactions on Information Theory*, vol. 41, no. 3, pp. 723–731, 1995. doi:10.1109/18.382018.
- [38] J. A. Storer and T. G. Szymanski, “Data compression via textual substitution,” *Journal of the ACM*, vol. 29, no. 4, pp. 928–951, 1982. doi:10.1145/322344.322346.
- [39] P. C. Shields, “Universal redundancy rates do not exist,” *IEEE Transactions on Information Theory*, vol. 39, no. 2, pp. 520–524, 1993. doi:10.1109/18.212281.
- [40] T. Bell, “A unifying theory and improvements for existing approaches to text compression,” Ph.D. dissertation, University of Canterbury, 1986. doi:10.26021/2003.
- [41] P. Fenwick, “Differential Ziv-Lempel text compression,” *Journal of Universal Computer Science*, vol. 1, no. 8, pp. 591–602, 1995. doi:10.3217/jucs-001-08-0591.
- [42] B. Ryabko, J. Astola, and K. Egiazarian, “Fast codes for large alphabets,” *Communications in Information and Systems*, vol. 3, no. 2, pp. 139–152, October 2003.

- [43] A. Said, “On the reduction of entropy coding complexity via symbol grouping: I—Redundancy analysis and optimal alphabet partition,” HP Laboratories, Tech. Rep. HPL-2004-145, August 2004.
- [44] T. M. Cover, “Enumerative source encoding,” *IEEE Transactions on Information Theory*, vol. 19, no. 1, pp. 73–77, January 1973. doi:10.1109/TIT.1973.1054929.
- [45] S. Jain and R. K. Bansal, “On match lengths, zero entropy and large deviations—with application to sliding window Lempel-Ziv algorithm,” arXiv:1411.1339, 2014.
- [46] L. A. Lastras-Montaña, “On certain pathwise properties of the sliding-window Lempel–Ziv algorithm,” *IEEE Transactions on Information Theory*, vol. 52, no. 12, pp. 5267–5283, 2006. doi:10.1109/TIT.2006.885458.
- [47] P. Ojala, “An optimizing hybrid LZ77 RLE data compression program,” pucrunch author documentation, updated November 22, 2008. <https://a1bert.kapsi.fi/Dev/pucrunch/>.
- [48] Linux kernel contributors, “LZO stream format as understood by Linux’s LZO decompressor,” kernel documentation, version-1 run-length extension. <https://www.kernel.org/doc/html/latest/staging/lzo.html>. Accessed September 12, 2026.
- [49] Y. Collet and contributors, “Zstandard API documentation,” `lib/zstd.h`, release v1.5.7, 2025. <https://github.com/facebook/zstd/blob/v1.5.7/lib/zstd.h>.
- [50] J. H. Reif and J. A. Storer, “Optimal encoding of non-stationary sources,” *Information Sciences*, vol. 135, nos. 1–2, pp. 87–105, 2001. doi:10.1016/S0020-0255(01)00103-7.
- [51] I. Kontoyiannis, P. H. Algoet, Yu. M. Suhov, and A. J. Wyner, “Nonparametric entropy estimation for stationary processes and random fields, with applications to English text,” *IEEE Transactions on Information Theory*, vol. 44, no. 3, pp. 1319–1327, 1998. doi:10.1109/18.669425.
- [52] Y. Gao, I. Kontoyiannis, and E. Bienenstock, “Estimating the entropy of binary time series: Methodology, some theory and a simulation study,” *Entropy*, vol. 10, no. 2, pp. 71–99, 2008. doi:10.3390/entropy-e10020071.
- [53] W. Hoeffding, “Probability inequalities for sums of bounded random variables,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, March 1963, doi:10.1080/01621459.1963.10500830.
- [54] A. Farruggia, P. Ferragina, A. Frangioni, and R. Venturini, “Bicriteria data compression,” arXiv:1307.3872, 2013.
- [55] P. A. Franaszek, L. A. Lastras-Montaña, S. Peng, and J. T. Robinson, “Data compression with restricted parsings,” in *Proc. Data Compression Conference (DCC)*, pp. 203–212, 2006. doi:10.1109/DCC.2006.22.
- [56] M. Plauth, J. Bruguera Micó, and A. Polze, “Improved data transfer efficiency for scale-out heterogeneous workloads using on-the-fly I/O link compression,”

- Concurrency and Computation: Practice and Experience*, vol. 35, no. 11, Art. e6101, 2023; first published December 1, 2020. doi:10.1002/cpe.6101.
- [57] Y. Collet and contributors, “LZ4 dictionary attachment and external dictionary implementation,” release v1.9.2, 2019, `lib/lz4.h`, `lib/lz4.c`, `lib/lz4hc.h`, and `lib/lz4hc.c`. <https://github.com/lz4/lz4/tree/v1.9.2/lib>.
- [58] J. Kärkkäinen, P. Sanders, and S. Burkhardt, “Linear work suffix array construction,” *Journal of the ACM*, vol. 53, no. 6, pp. 918–936, 2006. doi:10.1145/1217856.1217858.
- [59] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, “Linear-time longest-common-prefix computation in suffix arrays and its applications,” in *Combinatorial Pattern Matching (CPM 2001)*, LNCS, vol. 2089, pp. 181–192, 2001. doi:10.1007/3-540-48194-X_17.
- [60] M. A. Bender and M. Farach-Colton, “The LCA problem revisited,” in *LATIN 2000: Theoretical Informatics*, LNCS, vol. 1776, pp. 88–94, 2000. doi:10.1007/10719839_9.